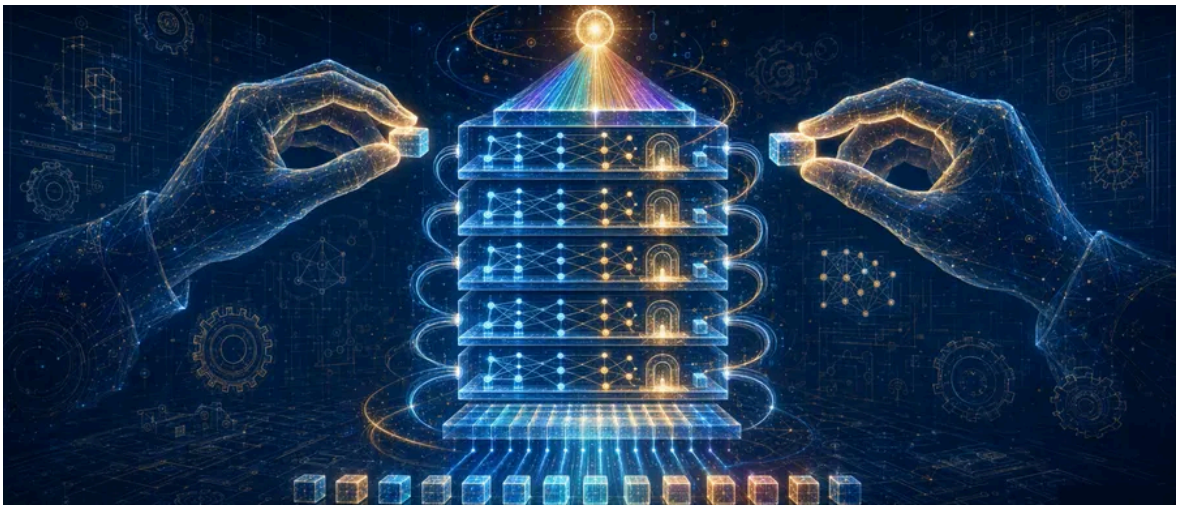
手把手带工程同学从零实现一个完整的 LLM（CS336）——Transformer 篇

原创 夏昊南 阿里技术 2026年10月14日 19:00 浙江



这是2026年的第 68 篇文章

（ 本文阅读时间：约 30 分钟 ）

01
前言

最近我在学习 CS336「Language Modeling from Scratch」，是斯坦福大学的大语言模型构建课程。这门课程在各种社交媒体（比如 X）中都很火爆，所以我想通过写系列文章的形式来分享这个学习过程。

虽然课程名字中叫「from Scratch」，但是它的含义是从零手搓，而不是从零教学，因此对于没有过神经网络/机器学习基础的同学来说，学习起来还是比较困难的，需要有较为完善的体系知识。它禁止我们使用 PyTorch 中的任何现有模块，全部需要从零开始构建（torch.nn.Parameter、torch.nn.Module、torch.optim.Optimizer 等除外）。

> 先叠个甲，我之前的背景是纯工程开发，大概在 25 年初开始转向 Agent 开发，此前也没有过系统的机器学习、神经网络学习经验，因此我学习 CS336 的过程也是逐渐查漏补缺，即用即学，如果专业的算法同学发现文章有不当之处，欢迎在评论区交流指正。

学习这门课程，最主要的就是完成它的 Assignment 作业，这一篇是这个系列中的第一篇文章，会跟随 CS336 Assignment1 的思路，完整介绍如何从零手搓一个 Transformer（注：非 Attention Is All You Need 原始论文版本，其中会引入 RoPE、SwiGLU 等变体，这也是 CS336 这门课程的优秀之处）。

> Assignment1 的原始资料：stanford-cs336/assignment1-basics

如果你也想学习 LLM 的底层原理，强烈建议跟着本篇文章，独立认真完成 Assignment 1（注：CS336 的 Assignment 中都会附带一份 AGENTS.md，以防你的 coding agent 直接帮你一键完成作业）。

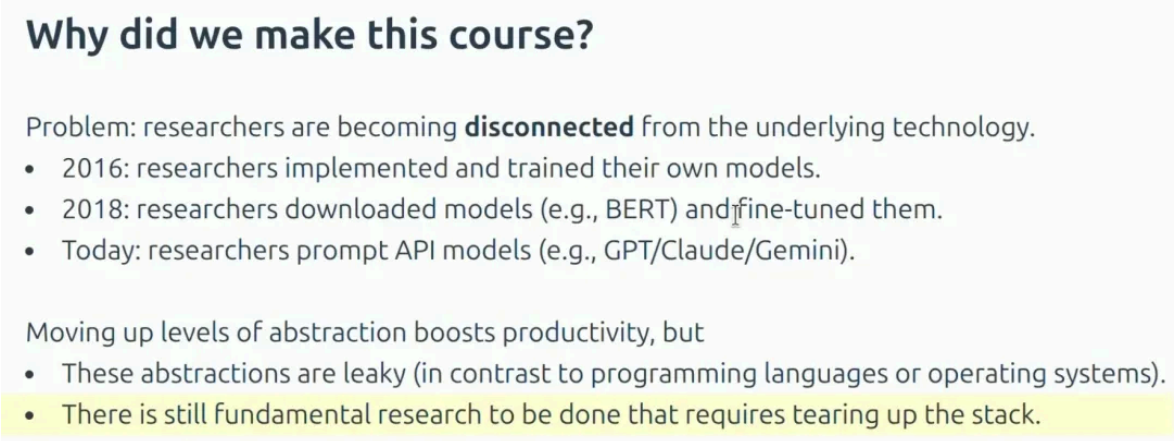
**为了先建立对于神经网络体感，文章开头会先从基础的 MLP 引入。*

02
为什么要学习这个

相信很多 Agent 开发者（包括我在内），都只是把 LLM 当做一个魔法黑盒，我们只负责调用它的 API，它就可以实现一切。

但是如果不去了解 LLM 的底层运行原理，那就无法科学地回答：什么样的 prompt 是更好的？新模型发布后的 model card 里面写的参数都是什么意思？technical report 里面的各种架构变动又代表着什么？更具体地说，最近很火的 Jev，和我们传统认知的小模型在架构上的本质区别是什么？为什么它能这么快？而且随着 AI 对于传统软件架构的冲击，越来越多的 Agent 工程同学同时也要为运行效果负责，必须要去了解模型原理、推理、训练。

附一张 CS336 课程讲义中的截图：



03
MLP

在开始 Transformer 之前，需要先了解一个简单的神经网络是什么样的。这里从 MLP（Multi-Layer Perceptron，多层感知机）开始，它是一种最基础的**前馈网络（Feed-Forward Network，FFN）**。“前馈”指数据从输入出发，依次经过各层计算得到输出，网络中没有形成循环的连接。

我们先用它做一个小实验，看看数据怎么输入、模型怎么得到预测结果，以及训练到底是在做什么。

线性回归

假设我们拿到 `(1, 3)、(2, 5)、(3, 7)` 三组输入和答案，希望程序根据这些数据找到规律，再去预测新的输入。

我们先尝试用一条直线拟合这些数据，模型写成 `y_pred = w * x + b`，w（权重）和 b（偏置）是可以调整的参数。根据数据不断调整它们，让预测结果更接近答案，这个过程就叫训练。

上面这种用直线拟合数据、预测数值的做法，叫线性回归。PyTorch 提供了一个现成的 **线性层** `nn.Linear`，帮我们保存权重、偏置并完成计算。这里用 `nn.Linear(1, 1)`，表示每个样本输入一个数、输出一个数，执行的就是 `w * x + b`。

下面代码里的 Tensor（张量），可以先理解为 PyTorch 用来存放数字、进行计算的多维数组。`[[1.0], [2.0], [3.0]]` 是一个 3 行 1 列的 Tensor，形状（shape）是 `(3, 1)`：每行一个样本，每个样本只有一个输入值。

```
import torch
from torch import nn
x = torch.tensor([[1.0], [2.0], [3.0]])  # 已知的输入
y = torch.tensor([[3.0], [5.0], [7.0]])  # 对应的答案
model = nn.Linear(1, 1)  # 创建模型，内部随机初始化 w 和 b
pred = model(x)  # Linear 内部用当前的 w、b 计算 w * x + b
print("训练前的预测：", pred)
```

运行到这里，我们只是用初始的 w、b 算了一次，模型还没有学习过，预测通常和 `[3, 5, 7]` 对不上。接下来就要比较预测和答案的差距，再调整参数。

输入有多个数时，Linear 会对它们加权求和，再加上偏置。后面讲 Transformer 时，我们会用矩阵乘法手搓这个过程。

什么是训练

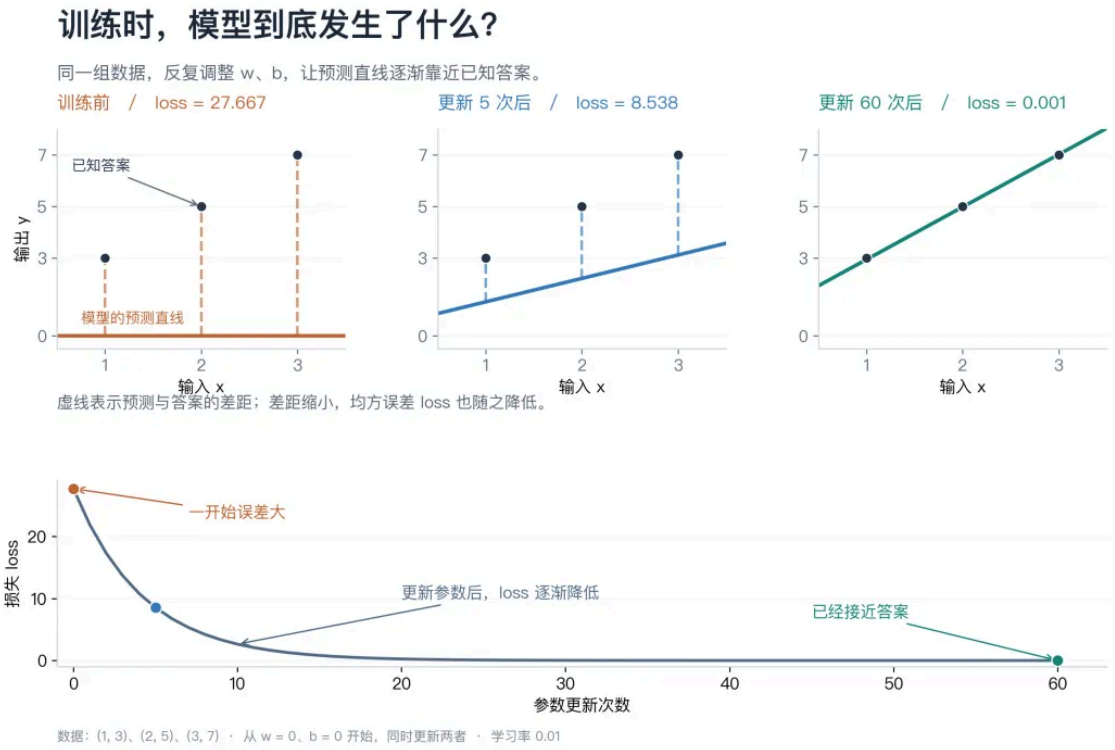
刚创建的 w、b 是随机初始化的，预测通常不准。我们需要用一个数衡量预测和答案的差距，这个数叫 **loss（损失）**。这里用均方误差（MSE），把每个样本的误差平方，再取平均：

```
答案：[3, 5, 7]
预测：[2, 4, 6]
loss = ((2-3)² + (4-5)² + (6-7)²) / 3 = 1
```

预测全部正确时，loss 就是 0。我们训练模型，就是调整 w、b，让这个数尽量变小。

那怎么知道参数该往哪调？梯度可以帮我们判断。直观上就像下坡，每次朝着 loss 更小的方向调整一点，这叫梯度下降。学习率控制步子大小，步子太大也会走过头。

看下面的例子：黑点是已知答案，彩色直线是模型的预测。随着 w、b 不断调整，直线逐渐靠近这些点，下面记录的 loss 也随之降低。



后面的 MLP 也是这个过程，只是需要调整的参数更多。

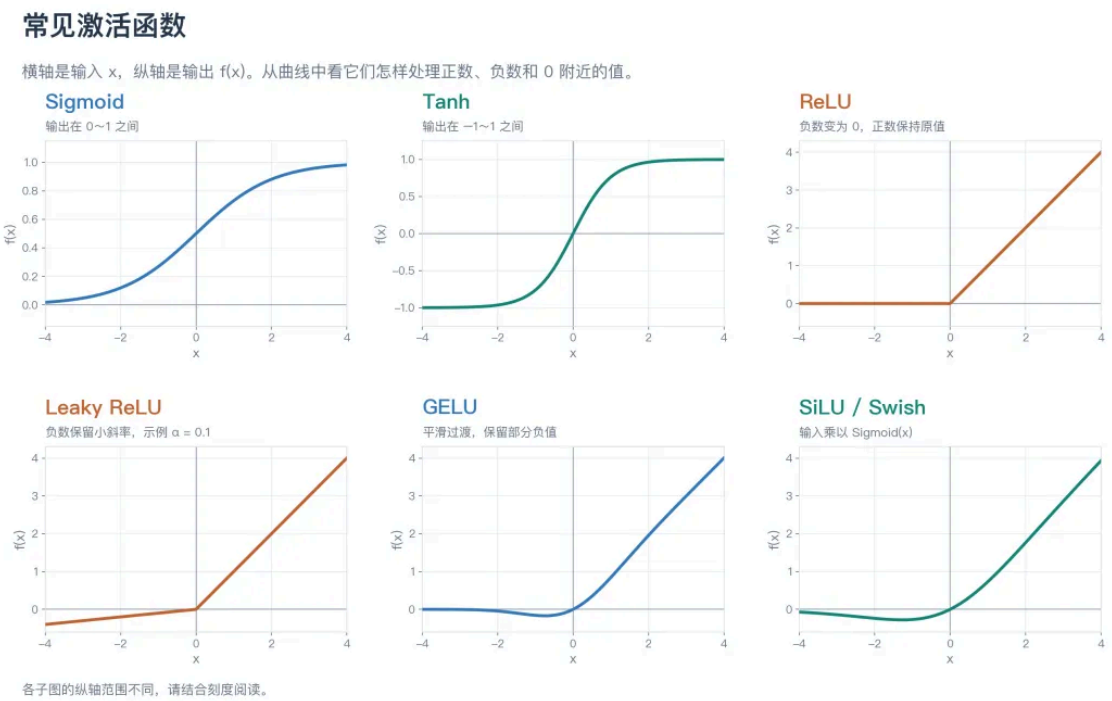
非线性拟合

前面我们用一个 Linear 拟合了一条直线。但实际任务中，输入和输出之间经常是更复杂的非线性关系，比如一条弯曲的曲线，一个 Linear 就不够用了。那把多个 Linear 叠起来行不行？我们把两层展开看看：

```
h = w1 * x + b1
y = w2 * h + b2
  = (w2 * w1) * x + (w2 * b1 + b2)
```

把上面的 `w2 * w1` 看成新的权重，`w2 * b1 + b2` 看成新的偏置，就又回到了 `w * x + b`。所以，**多个 Linear 直接串起来，仍然等价于一个 Linear**，这个结论对多个输入、输出也成立。这是线性代数里的基本性质：仿射变换复合后仍然是仿射变换（带偏置的 Linear 严格来说叫仿射变换）。

所以我们在 Linear 之间加入非线性的激活函数，让网络能拟合曲线。下面是几种常见激活函数的曲线，横轴是输入，纵轴是输出。我们这里先用 Tanh。




```python
# Sequential 按顺序执行，上一层的输出交给下一层
model = nn.Sequential(
    nn.Linear(1, 64),   # 一个输入，算出 64 个中间值
    nn.Tanh(),          # 对每个值做非线性变换
    nn.Linear(64, 64),  # 对这些值加权求和，加上偏置，得到 64 个输出
    nn.Tanh(),
    nn.Linear(64, 1),   # 最后得到一个预测值
)
```

这就是我们这里的 MLP：三个 Linear，中间加了两个 Tanh。`Linear(1, 64)` 可以理解为同时计算 64 个不同的 `w * x + b`，每个都有自己的参数；Tanh 本身没有需要训练的参数。

输入和最终输出之间的中间层叫作**隐藏层**。上面的网络从 1 个输入值出发，经过两层各有 64 个数的中间表示，最后得到 1 个预测值，所以它有两个隐藏层，每层的**宽度**都是 64。这里的“隐藏”指这些数由网络内部计算出来，训练数据只提供输入和目标答案，并不直接指定这些中间值。

接下来我们用 `y = sin(x) + 0.3 * cos(3x)` 加一点随机噪声生成数据，看看这个 MLP 能不能学出曲线的规律。

一个完整的 MLP 可以参考下面这段代码，已经写了详细的注释，如果有不懂的地方可以问问自己的 AI。

```python
from __future__ import annotations
import math
import random
import torch
```

```python
from torch import nn
def make_dataset(n: int = 2048) -> tuple[torch.Tensor, torch.Tensor]:
    # x 原本是一维向量，unsqueeze(1) 把它变成 (n, 1)。
    # 神经网络通常按二维 batch 输入理解数据：每一行是一个样本，每一列是一个特征。
    # Java 类比：原来像 double[n]，unsqueeze 后更像 double[n][1]。
    x = torch.linspace(-2 * math.pi, 2 * math.pi, n).unsqueeze(1)
    # randn_like(x) 生成和 x 形状相同的随机噪声。
    # 这里加噪声是为了让任务更像真实数据：真实数据通常不会完美落在一条函数曲线上。
    noise = 0.05 * torch.randn_like(x)
    # 这里造一个有规律但不完全干净的函数，让模型学 sin/cos 的组合。
    # x 是输入，y 是标准答案。训练时模型只看到 x，loss 会拿预测值和 y 比较。
    y = torch.sin(x) + 0.3 * torch.cos(3 * x) + noise
    return x, y
class TinyMLP(nn.Module):
    def __init__(self) -> None:
        # nn.Module 是所有 PyTorch 模型/层的基类。
        # super().__init__() 会初始化父类里负责登记参数、子模块等基础设施。
        super().__init__()
        # nn.Sequential 会按顺序执行这些层：
        # Linear 做仿射变换，Tanh 提供非线性，否则模型只能学直线。
        # Linear(1, 64)：每个样本输入 1 个数字，输出 64 维隐藏表示。
        # Linear(64, 1)：最后把 64 维隐藏表示压回 1 个预测值。
        self.net = nn.Sequential(
            nn.Linear(1, 64),
            nn.Tanh(),
            nn.Linear(64, 64),
            nn.Tanh(),
            nn.Linear(64, 1),
        )
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # 在 PyTorch 里，调用 model(x) 时，实际会转到 forward(x)。
        # 这有点像 Java 里某个框架约定你实现 handle/request 方法，然后框架负责调用
        return self.net(x)
def main() -> None:
    # 固定随机种子，方便你多次运行时看到接近的结果。
    torch.manual_seed(42)
    random.seed(42)
    x, y = make_dataset()
    # 简单切分：前 80% 做训练集，后 20% 做验证集。
    split = int(0.8 * len(x))
    x_train, y_train = x[:split], y[:split]
    x_val, y_val = x[split:], y[split:]
    model = TinyMLP()
    # optimizer 负责根据梯度更新模型参数；lr 是每次更新的步子大小。
    # model.parameters() 来自 nn.Module，会递归收集 self.net 里 Linear 的 weigh
    optimizer = torch.optim.AdamW(model.parameters(), lr=3e-3)
```

```python
    # MSELoss 衡量预测值和真实值的平方误差，适合这个回归任务。
    loss_fn = nn.MSELoss()
    print("shape check")
    print(f"x_train: {tuple(x_train.shape)}")
    print(f"y_train: {tuple(y_train.shape)}")
    print(f"prediction: {tuple(model(x_train[:8]).shape)}")
    print()
    batch_size = 128
    for step in range(1, 801):
        # 真实训练通常不会每一步都用完整训练集，而是抽一个 mini-batch。
        # 好处：计算更快，也让每步梯度带一点随机性，常常更容易训练。
        # 随机抽一批样本。idx 的 shape 是 (batch_size,)。
        idx = torch.randint(0, len(x_train), (batch_size,))
        # 前向传播：输入 x，得到预测 pred。
        # x_train[idx] 的 shape 是 (batch_size, 1)，对应一批样本。
        pred = model(x_train[idx])
        loss = loss_fn(pred, y_train[idx])
        # PyTorch 默认会累积梯度，所以每一步训练前要先清空旧梯度。
        optimizer.zero_grad()
        # 反向传播：从 loss 出发，计算每个参数的梯度。
        loss.backward()
        # 根据梯度真正更新参数。
        optimizer.step()
        if step == 1 or step % 100 == 0:
            # 验证时不需要梯度；no_grad 会省内存，也避免误把验证计算放进计算图。
            # 注意：验证集不参与 optimizer.step，只用于观察模型有没有泛化。
            with torch.no_grad():
                val_loss = loss_fn(model(x_val), y_val)
            print(
                f"step {step:04d} | "
                f"train_loss={loss.item():.5f} | "
                f"val_loss={val_loss.item():.5f}"
            )
    with torch.no_grad():
        # 拿几个没放进 batch 的点，看模型现在会输出什么。
        sample_x = torch.tensor([[-3.0], [0.0], [3.0]])
        sample_y = model(sample_x)
    print()
    print("sample predictions")
    for value, pred in zip(sample_x.squeeze().tolist(), sample_y.squeeze().to
        print(f"x={value:+.1f} -> y_hat={pred:+.4f}")
if __name__ == "__main__":
    main()
```

语言模型基础-TinyGPT

前面我们用 MLP，根据输入的 x 预测一个数。接下来换个任务：给模型一段文字，让它预测下一个字。比如输入「今天天气」，模型接着生成「很」，再把「今天天气很」作为输入，继续预测。这样反复进行，就能逐步生成一段文字。

这里我们用一个极简的小 demo——TinyGPT 来演示这个过程，下面是完整代码，已经写了详细的注释，可以自己本地跑一跑，如果有不懂的地方可以问问自己的 AI。

> 这个小 demo 的目的是为了构建对于语言模型的体感，后续会逐章拆解其中的所有模块

```python
from __future__ import annotations
import math
import torch
from torch import nn
import torch.nn.functional as F
TEXT = (
    "agents plan actions observe results and update context. "
    "language models predict tokens from previous tokens. "
    "attention lets each token read useful earlier tokens. "
) * 40
class CausalSelfAttention(nn.Module):
    def __init__(self, d_model: int, num_heads: int, block_size: int) -> Nor
        super().__init__()
        # 多头 attention 会把 d_model 平均分给每个 head。
        # d_model 是每个 token 的向量维度；block_size 才是最大上下文长度。
        assert d_model % num_heads == 0
        self.num_heads = num_heads
        self.head_dim = d_model // num_heads
        # qkv 一次性算出 query/key/value，减少三次线性层的样板代码。
        # 输入和输出 shape 都围绕 (batch_size, seq_len, d_model) 展开。
        self.qkv = nn.Linear(d_model, 3 * d_model)
        self.proj = nn.Linear(d_model, d_model)
        # register_buffer 注册的是“不是参数、但要跟着模型移动/保存”的 tensor。
        # mask 不需要训练，所以不用 nn.Parameter。
        self.register_buffer("mask", torch.tril(torch.ones(block_size, block
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x: (batch_size, seq_len, d_model)
        batch_size, seq_len, d_model = x.shape
        q, k, v = self.qkv(x).chunk(3, dim=-1)
        # 多头 attention 标准 shape: (batch_size, num_heads, seq_len, head_d
        # 多头不是多跑几个模型，而是把 d_model 切成几份，让不同 head 学不同关系。
        q = q.view(batch_size, seq_len, self.num_heads, self.head_dim).trans
        k = k.view(batch_size, seq_len, self.num_heads, self.head_dim).trans
        v = v.view(batch_size, seq_len, self.num_heads, self.head_dim).trans
```

```
        # q @ k^T 得到 token 之间的相似度分数。
        scores = q @ k.transpose(-2, -1) / math.sqrt(self.head_dim)
        # causal mask 保证当前位置不能看未来 token。
        scores = scores.masked_fill(self.mask[:seq_len, :seq_len] == 0, floa
        weights = F.softmax(scores, dim=-1)
        out = weights @ v
        # 把 heads 维度拼回 d_model，恢复 (batch_size, seq_len, d_model)。
        out = out.transpose(1, 2).contiguous().view(batch_size, seq_len, d_m
        return self.proj(out)
class Block(nn.Module):
    def __init__(self, d_model: int, num_heads: int, block_size: int) -> Nor
        super().__init__()
        # 一个 GPT block 通常由两块组成：
        # attention 负责 token 之间交流，MLP 负责每个 token 自己的非线性加工。
        self.ln1 = nn.LayerNorm(d_model)
        self.attn = CausalSelfAttention(d_model, num_heads, block_size)
        self.ln2 = nn.LayerNorm(d_model)
        self.mlp = nn.Sequential(
            # 常见设计会先把 hidden 扩到 4 倍，再压回原维度。
            # 这不是改变序列长度，而是改变每个 token 向量内部的维度。
            nn.Linear(d_model, 4 * d_model),
            nn.GELU(),
            nn.Linear(4 * d_model, d_model),
        )
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # Pre-LN Transformer block：先 LayerNorm，再 attention/MLP。
        # 残差连接 x + f(x) 保持 shape 不变，也让梯度更容易流动。
        # 第一行：每个 token 先从上下文里读信息。
        x = x + self.attn(self.ln1(x))
        # 第二行：每个 token 再独立过一段 MLP 做加工。
        x = x + self.mlp(self.ln2(x))
        return x
class TinyGPT(nn.Module):
    def __init__(self, vocab_size: int, block_size: int) -> None:
        super().__init__()
        d_model = 64
        self.block_size = block_size
        # token_embedding 负责“这个 token 是什么”。
        # 输入 idx 是整数 id；embedding 输出连续向量。神经网络只能处理数值张量。
        self.token_embedding = nn.Embedding(vocab_size, d_model)
        # position_embedding 负责“这个 token 在第几个位置”。
        # 如果没有位置信息，attention 本身不天然知道顺序。
        self.position_embedding = nn.Embedding(block_size, d_model)
        self.blocks = nn.Sequential(
            Block(d_model, num_heads=4, block_size=block_size),
            Block(d_model, num_heads=4, block_size=block_size),
```

```
        )
        self.ln = nn.LayerNorm(d_model)
        self.lm_head = nn.Linear(d_model, vocab_size)
    def forward(self, idx: torch.Tensor, targets: torch.Tensor | None = None
        # idx: (batch_size, seq_len)，里面是 token ids。
        batch_size, seq_len = idx.shape
        # positions 是 0..seq_len-1，对应序列里的每个位置。
        # shape 是 (seq_len,)，加到 token_embedding 时会自动 broadcast 到 batc
        positions = torch.arange(seq_len, device=idx.device)
        # token 信息和位置信息相加，得到每个位置的初始表示。
        x = self.token_embedding(idx) + self.position_embedding(positions)
        x = self.blocks(x)
        x = self.ln(x)
        # logits: (batch_size, seq_len, vocab_size)，每个位置预测下一个 token。
        # 对某个位置来说，vocab_size 个数字就是“下一个 token 是词表中每个 id 的分
        logits = self.lm_head(x)
        loss = None
        if targets is not None:
            # cross_entropy 要求 (N, classes) 和 (N,)，所以合并 batch_size/seq
            loss = F.cross_entropy(logits.view(batch_size * seq_len, -1), ta
        return logits, loss
    @torch.no_grad()
    def generate(self, idx: torch.Tensor, steps: int) -> torch.Tensor:
        # generate 是推理阶段：不再给 targets，也不算 loss，只反复预测下一个 toke
        for _ in range(steps):
            # 如果序列超过 block_size，只保留最后 block_size 个 token 作为上下文
            context = idx[:, -self.block_size :]
            logits, _ = self(context)
            # 只取最后一个位置的 logits，因为生成时只需要预测“下一个 token”。
            probs = F.softmax(logits[:, -1, :], dim=-1)
            next_id = torch.multinomial(probs, num_samples=1)
            idx = torch.cat([idx, next_id], dim=1)
        return idx
def make_data():
    # 字符级 tokenizer：简单但足够演示语言模型训练。
    # 后面如果换成 BPE tokenizer，这里 data 的构造方式会变，但模型仍然吃 token id
    chars = sorted(set(TEXT))
    stoi = {ch: i for i, ch in enumerate(chars)}
    itos = {i: ch for ch, i in stoi.items()}
    data = torch.tensor([stoi[ch] for ch in TEXT], dtype=torch.long)
    return data, stoi, itos
def get_batch(data: torch.Tensor, batch_size: int, block_size: int):
    # 每个样本是一段连续 token ids。x 是输入上下文，y 是每个位置的下一个 token。
    starts = torch.randint(0, len(data) - block_size - 1, (batch_size,))
    x = torch.stack([data[i : i + block_size] for i in starts])
    # y 是 x 的下一个字符序列，用来做 next-token prediction。
```

```
    y = torch.stack([data[i + 1 : i + block_size + 1] for i in starts])
    return x, y
def decode(ids: torch.Tensor, itos: dict[int, str]) -> str:
    return "".join(itos[i] for i in ids.tolist())
def main() -> None:
    torch.manual_seed(123)
    # block_size 是模型一次最多能看的上下文长度。
    block_size = 32
    data, stoi, itos = make_data()
    model = TinyGPT(vocab_size=len(stoi), block_size=block_size)
    optimizer = torch.optim.AdamW(model.parameters(), lr=3e-3)
    x, y = get_batch(data, batch_size=4, block_size=block_size)
    logits, loss = model(x, y)
    assert loss is not None
    print("shape check")
    print(f"x: {tuple(x.shape)}")
    print(f"logits: {tuple(logits.shape)}")
    print(f"loss: {loss.item():.4f}")
    print()
    for step in range(1, 301):
        x, y = get_batch(data, batch_size=32, block_size=block_size)
        _, loss = model(x, y)
        assert loss is not None
        # 标准训练三步：清梯度 -> 反向传播 -> 更新参数。
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        if step == 1 or step % 50 == 0:
            print(f"step {step:04d} | loss={loss.item():.4f}")
    start = torch.tensor([[stoi["a"]]], dtype=torch.long)
    sample = model.generate(start, steps=180)[0]
    print()
    print(decode(sample, itos))
if __name__ == "__main__":
    main()
```

预测下一个字

神经网络无法直接对文字做计算，所以我们先给字符编号，把一段文字转换成一串数字。上面代码中的 `make_data()` 就是在执行这一过程，比如：

```
字符：a  g  e  n  t  s
编号：2  8  6  14  19  18
"agents" → [2, 8, 6, 14, 19, 18]
```

这份字符和编号的对应表叫词表。这里按字符编号，相当于一个简化的 Tokenizer（分词器），每个字符就是一个 token。实际语言模型常用 BPE 等分词方式，一个 token 可以包含多个字符，后面会详细介绍。

前面的 MLP 要预测一个具体数值，输出的就是数值。这里要从词表里选择下一个字符，属于分类任务。同一段文字后面也可以有不同的接法，所以我们用概率表示各个字符出现的可能性。

模型先给词表里的每个字符输出一个分数，这组分数叫 **logits**。再通过 softmax，把它们转换成总和为 1 的概率。softmax 的做法是先对每个分数取指数，再除以这些指数值的总和，原来的分数越高，得到的概率就越大。

$$p_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

这里 z_i 是第 i 个字符的分数，p_i 是它对应的概率，分母是所有字符的分数取指数后的总和。

上面代码中的 `generate()` 就是在执行这一过程。比如输入 `agent` 后，各个字符的概率可以是这样（仅作示意）：

下一个字符	概率
s	60%
空格	25%
其余字符合计	15%

模型认为 `s` 更有可能接在后面，拼成 `agents`。文字生成时，我们就可以根据这些概率选出下一个字符。

完整过程可以先记成：

```
文字 → 字符编号 → TinyGPT → 每个字符的分数 → 概率 → 选出下一个字符
```

在 `TinyGPT.forward()` 里，`token_embedding` 会根据编号查出一组可以训练的数，这一步叫 Embedding。然后用 Transformer 处理前面的文字，最后通过 `lm_head` 这个 Linear，输出词表中每个字符的分数。这里又用到了前面见过的线性层，只是输出从一个数变成了一组数。

Attention

输入 `agent`，预测后面的字符时，模型需要结合前面的 `a、g、e、n、t`。这些字符的信息怎么汇集到一起？这里就要用到 Attention（注意力机制）。

前面每个字符已经通过 Embedding 转成了一组数。Attention 会为当前位置能看到的各个位置计算权重，再按权重把它们的信息加起来。权重越大，对当前位置的输出贡献就越大。上面代码

中的 `CausalSelfAttention` 就是在执行这一过程。

比如 `t` 所在的位置可以读取 `a、g、e、n、t` 的信息，用来预测下一个字符；`a` 所在的位置只能读取自己。这个限制叫因果遮罩（causal mask），避免模型在训练时提前看到后面的答案。

前面我们介绍过，MLP 是一种前馈网络（FFN）。这里的 TinyGPT 使用 `Linear → GELU → Linear` 组成的 MLP 作为**前馈模块**，对应代码中的 `self.mlp`。它先把每个 token 的向量扩展到原来的 4 倍大小，经过 GELU 激活后，再变回原来的大小。

在 `Block.forward()` 中，数据先经过 Attention 汇集上下文信息，再经过这个前馈模块做非线性处理：

```
x = x + self.attn(self.ln1(x))
x = x + self.mlp(self.ln2(x))
```

这里的 MLP 对每个位置分别计算，位置之间的信息交流由 Attention 完成。

这里的"位置"就是字符在序列里的下标。比如 `agent` 中，`a` 在位置 0，`g` 在位置 1，依次类推。TinyGPT 除了查字符的 Embedding，还会用这个下标查 `position_embedding`，把两个向量相加后送进 Transformer。同一个字符出现在不同位置时，就会得到不同的初始表示。

具体的 attention 机制会在后面详细介绍。

训练

模型刚创建时参数是随机的，自然不知道 `agent` 后面应该接什么。训练需要给它输入和对应的答案，但文字里的答案从哪里来？

原文的下一个字符就是答案。 比如训练文本开头的 `agents`，就可以构造这些训练目标：

```
看到 a     → 预测 g
看到 ag    → 预测 e
看到 age   → 预测 n
看到 agen  → 预测 t
看到 agent → 预测 s
```

`get_batch()` 把输入和答案错开一个位置，就能同时准备好这些目标。把编号还原成字符看，就是：

```
输入 x: a  g  e  n  t
答案 y: g  e  n  t  s
```

训练时，模型会在每个位置预测下一个字符。前面介绍的因果遮罩保证每个位置只能利用自己和前面的内容。

前面的 MLP 用均方误差衡量预测值和答案差多少。这里模型输出的是一组字符分数，我们用**交叉熵损失（cross-entropy loss）**衡量它预测得好不好。

比如输入 `agent`，原文的下一个字符是 `s`。模型给 `s` 的概率只有 10% 时，说明它不太看好这个正确答案，loss 就比较大；如果给到 90%，loss 就比较小。训练会根据这个误差调整参数，让模型给正确答案更高的概率。

上面代码中的 `F.cross_entropy` 就是在执行这一过程，直接传入 logits 和答案编号即可，它内部会处理从分数到概率的计算。接着回到 `main()` 的训练循环，反向传播、更新参数。只不过这次训练的是 TinyGPT 里面的参数，包括 Embedding、Attention 和 MLP 等模块。

开头的 `TEXT` 用下面三句英文重复 40 遍作为训练文本：

```
agents plan actions observe results and update context.
language models predict tokens from previous tokens.
attention lets each token read useful earlier tokens.
```

模型有两层 Transformer Block，一次最多看 32 个字符，每次训练抽取 32 段文本。我们可以运行一下观察日志：

```
step 0001 | loss=3.3545
step 0050 | loss=0.9013
step 0100 | loss=0.1608
step 0300 | loss=0.1020
```

loss 整体降下来了，说明模型在这些训练文本上，预测下一个字符的能力变得更强了。

推理与生成

训练完成后，我们用模型预测新的输入，这个过程就是推理。每轮推理都根据当前文本预测下一个 token，再通过采样选出一个 token，追加到文本末尾。`generate()` 把这个过程循环执行，就能逐步生成一段文字。



`main()` 最后从字符 `a` 开始，调用 `model.generate()` 连续生成 180 个字符，再由 `decode()` 把编号还原成文字，运行后可以得到结果：

```
anguage models predich tokens from previous tokens. attention lets each token
 earlier tokens. agents plan actions observe results and update context. langu
```

已经能看到训练文本里的句子了，但也有 `predich` 这样的拼写错误。这个模型只见过几句反复出现的文字，训练 loss 降低还不足以说明它能处理其他文本。通过这个示例，我们可以看到语言模型如何从文字中获得训练目标，又如何通过反复预测生成文字。

接下来就沿着刚才的数据流，从 Tokenizer 开始，把各个模块拆开实现。这个 demo 用了 PyTorch 自带的 Linear、Embedding、LayerNorm 等组件，正式进入 Assignment 1 后，我们会手写相应组件，并用上 RMSNorm、RoPE 和 SwiGLU。

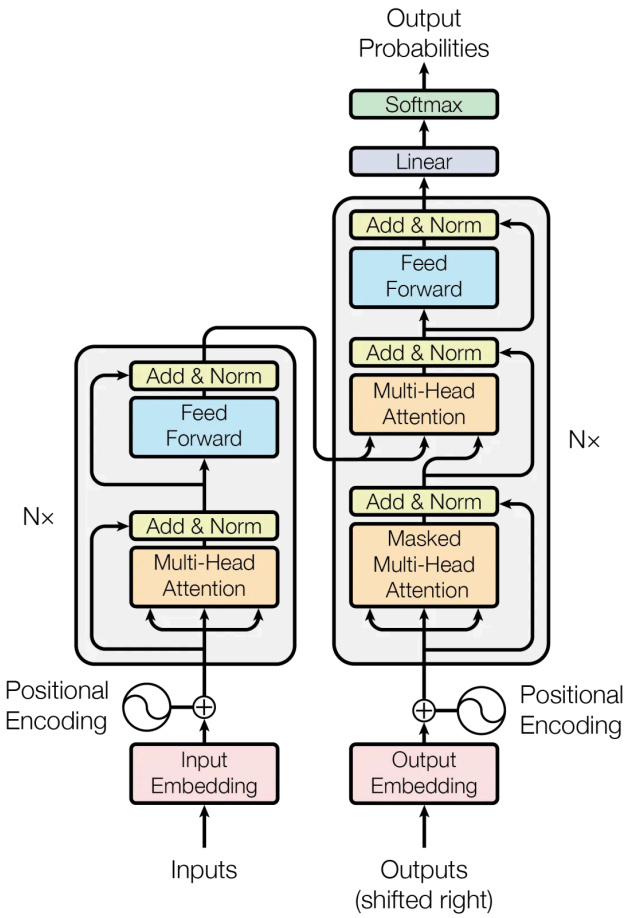
05
拆解 Transformer

这一节我会用我自己的 CS336 Assignment1 的实现来对照着逐一拆解 Transformer 的模块，我的实现在这里：cs336-assignment。后面的代码片段全都出自这个仓库。

> 如果你也想尝试一下 Assignment1，强烈建议停止阅读，去 clone 原始仓库：stanford-cs336/assignment1-basics，自己独立完成。如果你只是想了解一下 Transformer 的各个模块，那就让我们继续吧！

5.1 Architecture

前面我们已经跑通了一个小型语言模型，接下来开始拆解它的内部结构。先看一下《Attention Is All You Need》论文中最初的 Transformer 架构：



图片来源：Attention Is All You Need，Figure 1。

图中左边是 **Encoder（编码器）**，右边是 **Decoder（解码器）**。原论文主要用它做机器翻译。Encoder 先处理原文，Decoder 再结合 Encoder 的输出和已经生成的译文，逐个预测后面的词。

现在常见的 GPT、Llama 这类生成式大语言模型采用的是 **decoder-only** 架构。已有文本放在同一条序列里，每个位置只能读取自己和前面的内容，再预测下一个 token。它省去了独立的 Encoder，以及 Decoder 中读取 Encoder 输出的那层 Attention。前面的 TinyGPT 也是这个结构。

我们在 CS336 Assignment 1 中要实现的，可以理解为一个 **Llama 风格的简化语言模型**，采用 decoder-only 结构，并使用 RMSNorm、RoPE 和 SwiGLU。这些也是 Llama 架构 中的设计，后面会逐个介绍。下面是作业给出的架构图：

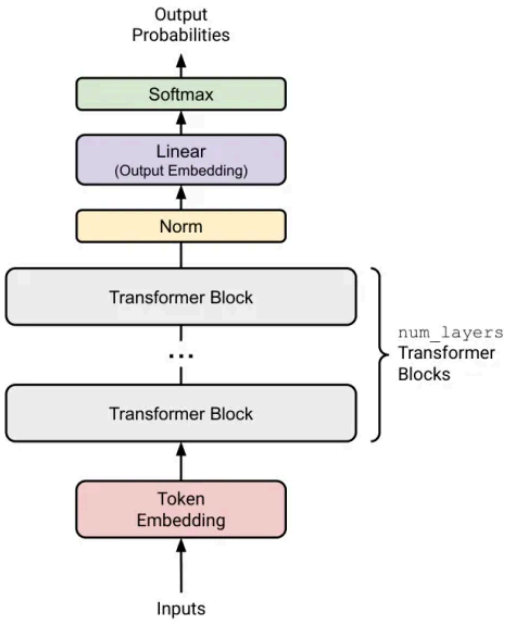


Figure 1: An overview of our Transformer language model.

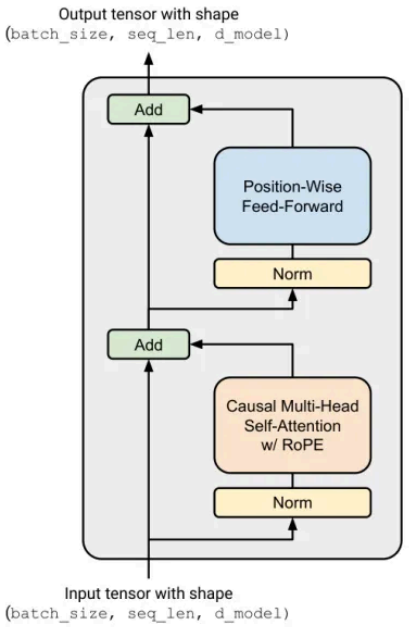


Figure 2: A pre-norm Transformer block.

图片来源：CS336 Assignment 1，Figure 1、Figure 2。

沿着左图从下往上看，token 编号先经过 **Embedding** 转成向量，再经过多层 **Transformer Block**，最后通过 Norm 和 Linear 得到词表中每个 token 的分数。用 softmax 把分数转成概率，就接上了前面讲的预测过程。

右图把一个 Block 展开了。其中 **Attention** 让各个位置读取上下文，**Feed-Forward** 是对每个位置单独计算的前馈网络，这里使用 SwiGLU。图中的 Norm 使用 RMSNorm，放在这两个模块之前，所以叫 **pre-norm**；绕过模块、连到 Add 的线表示把模块的输入加回输出，这叫**残差连接**。

接下来就按这张图拆开实现。先从模型输入之前的文本编码与 Tokenizer 开始。

5.2 Tokenizer 与 Embedding

前面的 TinyGPT 给每个字符编了一个号。这里我们换成 Assignment 1 要求的 **byte-level BPE**。它会先把文字变成字节，再把经常一起出现的字节组合成一个 token。

字符、Unicode 与 UTF-8

先来复习一下字符编码。我们看到的 `A`、`中` 都是字符，**Unicode 给这些字符分配了统一的编号**，这个编号叫作码点。比如 `A` 的编号是 65，`中` 的编号是 20013。

但文字在文件里存储、在网络上传输时，用的是字节。一个字节有 8 位，能表示的整数范围是 0～255，像 20013 这样的编号，一个字节就放不下了。**UTF-8 规定了怎么把一个 Unicode 码点编码成字节**，根据码点的不同，会用到 1～4 个字节。

字符	Unicode 码点（十进制）	UTF-8 字节（十进制）	字节数
`A`	65	`[65]`	1
`中`	20013	`[228, 184, 173]`	3

所以，一个字符不一定只占一个字节。`A中` 只有两个字符，经过 UTF-8 编码后却有四个字节。Python 中的 `encode()` 和 `decode()` 就能完成这两个方向的转换。

```
>>> text = "A中"
>>> data = text.encode("utf-8")
>>> list(data)
[65, 228, 184, 173]
>>> data.decode("utf-8")
'A中'
```

把文字转成 UTF-8 字节后，每个字节都只会落在 0～255 之间。因此，初始词表只要为这 256 种字节各准备一个 token，就能表示任意有效的 UTF-8 文本，也能处理训练时没见过的字符。

以完整的字符串 `我爱AI！` 为例，逐个字符转成 UTF-8 字节，会得到下面的结果。

字符	UTF-8 字节
我	`[230, 136, 145]`
爱	`[231, 136, 177]`
A	`[65]`
I	`[73]`
！	`[239, 188, 129]`

```
"我爱AI！"
→ [230, 136, 145, 231, 136, 177, 65, 73, 239, 188, 129]
```

原来的 5 个字符变成了 11 个 token。只按字节切分，序列会比较长。接下来，BPE 会把经常相邻出现的 token 合并成更长的字节片段，让同一段文字可以用更少的 token 表示。

BPE

BPE 每轮统计相邻 token 对出现的次数，把最高频的一对合成一个新 token。反复执行，直到词表达到指定大小，或者已经没有可以合并的相邻对。

假设训练文本里有四种单词，`low` 出现 5 次，`lower` 出现 2 次，`widest` 出现 3 次，`newest` 出现 6 次。这些英文字母在 UTF-8 中都只占一个字节，所以初始时每个字母就是一个

token，比如 `newest` 会拆成 `n e w e s t`。

`widest` 和 `newest` 都有相邻的 `s t`，加起来出现了 3 + 6 = 9 次。它是出现次数最多的组合之一，我们的实现会在同频时选择字典序更大的那一对，因此第一轮选中 `s t`，把它合成一个新 token `st`。此时 `newest` 变成 `n e w e st`。重新统计后，下一轮再把相邻的 `e st` 合成 `est`，于是变成 `n e w est`，从最初的 6 个 token 缩短到 4 个。

每个方框表示一个 token，下面这张图展示了前两轮的频次统计和合并结果。



前面的例子里，我们直接拿单词来统计。实际输入是一整段文字，需要先按规则切成若干片段，再分别做 BPE，这一步叫**预分词**。比如，我们的正则规则会把 `Hello world!` 切成下面这样。

```
"Hello world!" → ["Hello", " world", "!"]
```

其中 `" world"` 保留了开头的空格。每个片段随后转成字节，BPE 的频次统计和合并都只在片段内部进行。比如 `Hello` 里的 `l` 和 `o` 可以合并，但末尾的 `o` 不能和下一个片段开头的空格合并。这样就给合并划定了边界，避免跨片段组合不断进入词表。

预分词得到的片段还会继续拆分、合并，最终一个片段可以对应一个或多个 token。代码里的 `PATTERN` 和 `regex.findall()` 就是在执行预分词。`<|endoftext|>` 这样的特殊 token 会提前单独切出，作为一个完整 token 保留，不参与普通文本的合并。

下面是完整实现，前面讲的内容就是 `train_bpe()`，入参是训练文本的文件路径、目标词表大小和特殊 token 列表，出参是 `vocab` 和 `merges`。

`vocab` 负责保存 token 编号与字节串的对应关系，`merges` 保存有顺序的合并规则。

```python
import regex
from collections import Counter
PATTERN = r"""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?
def train_bpe(input_path, vocab_size, special_tokens):
    # 以 bytes 读取语料，之后显式按 UTF-8 解码，避免默认编码影响结果。
    with open(input_path, "rb") as f:
        corpus = f.read()
    text = corpus.decode("utf-8")
    # special token 是文档边界：不把它的内部字符交给预分词或 BPE 统计。
    # 这里先按照 special 把原始文本分开成 spilt list
    spilt_list = []
    if len(special_tokens) == 0:
        spilt_list.append(text)
    else:
        special_token_pattern = "|".join([regex.escape(special) for special i
        spilt_list = regex.split(special_token_pattern, text)
    # 再对拆开来的每一项，按照作业里面的预分词器（实际上就是一段正则）进行预分词
    # 为什么要预分词，可以看这个：https://zhuanlan.zhihu.com/p/692508797
    all_pretokens = []
    for spilt in spilt_list:
        all_pretokens.extend(regex.findall(PATTERN, spilt))
    # 先统计下预分词后，每个词的频率，例如 "low" -> 5。
    pretoken_text_counts = Counter(all_pretokens)
    # 将每个 pre-token 从字符串转为 utf-8 的 bytes，例如 "low"-> b"low"
    # 然后再拆成独立的
    # token_sequence_counts 里面的内容：(b"l", b"o", b"w") -> 5
    token_sequence_counts = {}
    for key, value in pretoken_text_counts.items():
        raw_bytes = key.encode("utf-8")
        bytes_tokens = []
        # bytes 实际上是 int 的数组，所以这里取出来的是 int，还需要手动创建一个 byt
        for byte_int in raw_bytes:
            bytes_tokens.append(bytes([byte_int]))
        token_sequence_counts[tuple(bytes_tokens)] = value
    # 初始词表包含全部 256 个可能的单 byte 值。
    # special token 作为一个完整 token 加入 vocab，不拆成内部的 byte token。
    merges = []
    vocab = {}
    for token_id in range(256):
        vocab[token_id] = bytes([token_id])
    for token_id, special_token in enumerate(special_tokens, start=256):
        vocab[token_id] = special_token.encode("utf-8")
    # 每轮只学习一种最高频 pair；新 token 会让 vocab 的大小增加 1。
    current_vocab_size = len(vocab)
    while current_vocab_size < vocab_size:
        # 基于「当前」token 序列统计相邻 pair 的加权次数。
```

```
        # 例如某序列出现 5 次，其中的每个相邻 pair 都贡献 5 次，而不是 1 次。
        pair_counts = Counter()
        for seq, count in token_sequence_counts.items():
            for pair in zip(seq, seq[1:]):
                pair_counts[pair] += count
        # 没有相邻 pair 时，说明无法继续学习新的 merge。
        if not pair_counts:
            break
        # 找最高频 pair；同频时由 find_max 按字典序选择更大的 pair。
        pair, count = find_max(pair_counts)
        # 两个 bytes token 拼接成一个新 bytes token，例如 b"l" + b"o" -> b"lo"。
        target = pair[0] + pair[1]
        vocab[current_vocab_size] = target
        # merges 必须保留创建顺序；编码时会按这个顺序应用规则。
        merges.append(pair)
        # 不能边遍历边修改旧表：读取旧 token_sequence_counts，
        # 将 merge 后的结果写入新表，全部完成后再整体替换。
        new_token_sequence_counts = {}
        for seq, count in token_sequence_counts.items():
            new_seq = []
            index = 0
            while index < len(seq):
                if index + 1 < len(seq) and (seq[index], seq[index + 1]) == p
                    # 命中时消费两个旧 token，写入一个新 token；因此不会重叠合并。
                    new_seq.append(target)
                    index += 2
                else:
                    # 未命中时保留当前 token，继续检查下一个位置。
                    new_seq.append(seq[index])
                    index += 1
            new_token_sequence_counts[tuple(new_seq)] = count
        token_sequence_counts = new_token_sequence_counts
        # 本轮新增了一个 vocab token，下一轮将基于更新后的序列重新统计 pair。
        current_vocab_size += 1
    return vocab, merges
def find_max(pair_counts):
    return max(pair_counts.items(), key=lambda item: (item[1], item[0]))
```

Tokenizer

前面通过 BPE 得到了词表和合并规则。接下来把它们封装成一个 Tokenizer，提供 `encode()` 和 `decode()` 两个方法，分别把文字转换成 token 编号，以及把编号还原成文字。

```
import regex
```

```python
from collections.abc import Iterable, Iterator
PATTERN = r"""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(
class Tokenizer:
    def __init__(
        self,
        vocab: dict[int, bytes],
        merges: list[tuple[bytes, bytes]],
        special_tokens: list[str] | None = None
    ):
        if special_tokens is None:
            special_tokens = []
        self.vocab = vocab
        self.merges = merges
        self.special_tokens = special_tokens
        # special token 最好校验一下，如果不在 vocab 里面，我们就给他手动加一下然
        for sp_token in special_tokens:
            sp_token_bytes = sp_token.encode("utf-8")
            if(sp_token_bytes not in self.vocab.values()):
                self.vocab[max(self.vocab.keys()) + 1] = sp_token_bytes

        # 一个字典推导式，实际上就是把 vocab 的 kv 调转一下，因为它是 int2bytes 的
        self.vocab_b2i = {value: key for key, value in self.vocab.items()}
    def encode(self, text: str) -> list[int]:
        token_int_list = []

        # 和 train_bpe 一样，还是先按照 special tokens 切分一下
        if len(self.special_tokens) == 0:
            split_list = [text]
        else:
            special_token_pattern = "|".join(
                regex.escape(special)
                for special in sorted(self.special_tokens, key=len, reverse=
            )
            split_list = regex.split(f"({special_token_pattern})", text)

        # 切分后的 split_list: ["Hi", "<|endoftext|>", " there"]
        # 对齐进行遍历，对每一个单独的结果进行 encode
        for part in split_list:
            if(part == ""):
                continue

            # 如果是 special token，就直接从 vocab 里面找到 id，然后 append 到最
            if(part in self.special_tokens):
                token_int_list.append(self.vocab_b2i[part.encode("utf-8")])
            else:
                # 否则就是普通的正常文本
```

```
                # 先预分词，Tokenizer 的 encode 也是不能跨 pre-token 的
                # pre_tokens_bytes_list 的内容：[[b"a", b"b", b"d"],[b"a", b"
                pre_tokens = regex.findall(PATTERN, part)
                pre_tokens_bytes_list = []
                for pre_token in pre_tokens:
                    raw_bytes = pre_token.encode("utf-8")
                    pre_tokens_bytes = []
                    for pre_token_byte_int in raw_bytes:
                        pre_tokens_bytes.append(bytes([pre_token_byte_int]))
                    pre_tokens_bytes_list.append(pre_tokens_bytes)

                # 这里三层循环
                current_list_index = 0
                while current_list_index < len(pre_tokens_bytes_list):
                    # 1. 先依次取出预分词的结果，取出来的内容 pre_token_bytes: [
                    pre_token_bytes = pre_tokens_bytes_list[current_list_ind
                    for merge in self.merges:
                        # 2. 遍历 merges，一条 merge 规则的输入是一条 token 序
                        new_bytes = []
                        i = 0
                        while i < len(pre_token_bytes) - 1:
                            # 3. 从左到右扫描最新 token 序列，根据 merge 关系，
                            if merge == (pre_token_bytes[i], pre_token_bytes
                                new_bytes.append(pre_token_bytes[i]+pre_toke
                                i += 2
                            else:
                                new_bytes.append(pre_token_bytes[i])
                                i += 1
                        # 从左到右扫描 完 token 序列后，边界要处理下
                        # 例如 pre_token_bytes = [b"a", b"b", b"c"] merge =
                        # merge 完了之后还剩最后一个 b"c" 就退出了，要手动添加到
                        if(i == len(pre_token_bytes) - 1):
                            new_bytes.append(pre_token_bytes[i])
                        # 新的 token 序列需要赋值，进入下一个 merge 规则的循环
                        pre_token_bytes = new_bytes

                    # 2、3循环跑完后，pre_token_bytes 就是一个已经完整应用过所有
                    for pre_token in pre_token_bytes:
                        token_int_list.append(self.vocab_b2i[pre_token])

                    current_list_index += 1

        return token_int_list

    def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]:
        for part in iterable:
```

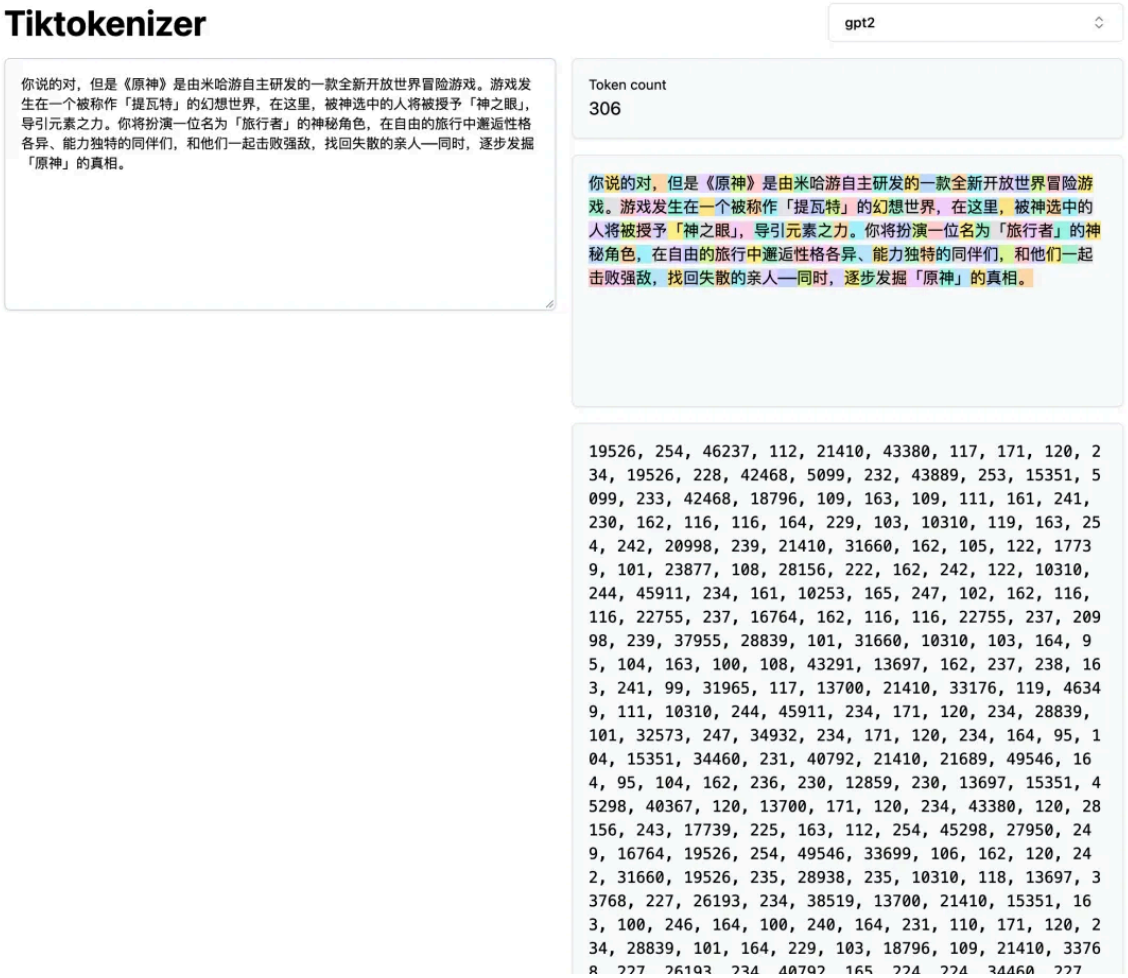
```
            for token_id in self.encode(part):
                yield token_id

    def decode(self, ids: list[int]) -> str:
        bytes_list = []
        for token_id in ids:
            bytes_list.append(self.vocab[token_id])
        return b"".join(bytes_list).decode("utf-8", errors = "replace")
```

Tokenizer 在词表大小和序列长度之间做取舍。合并常见组合可以缩短序列，但词表越大，模型的存储和输出计算开销也越大。

BPE 学出词表和合并规则后，编码就按这套规则执行。**训练和推理要使用同一套 Tokenizer**，保证同一个编号始终对应相同的内容。

推荐一个网站：Tiktokenizer，可以直观地体验分词结果，这里我们演示 gpt2 的 tokenizer：



Embedding

Tokenizer 把文字转换成了 token 编号，但编号本身不能表达语义关系。假设「国王」和「皇后」各是一个 token，编号分别是 `42` 和 `87`，这两个数的大小和差距，并不能告诉模型它们的含义有多接近。

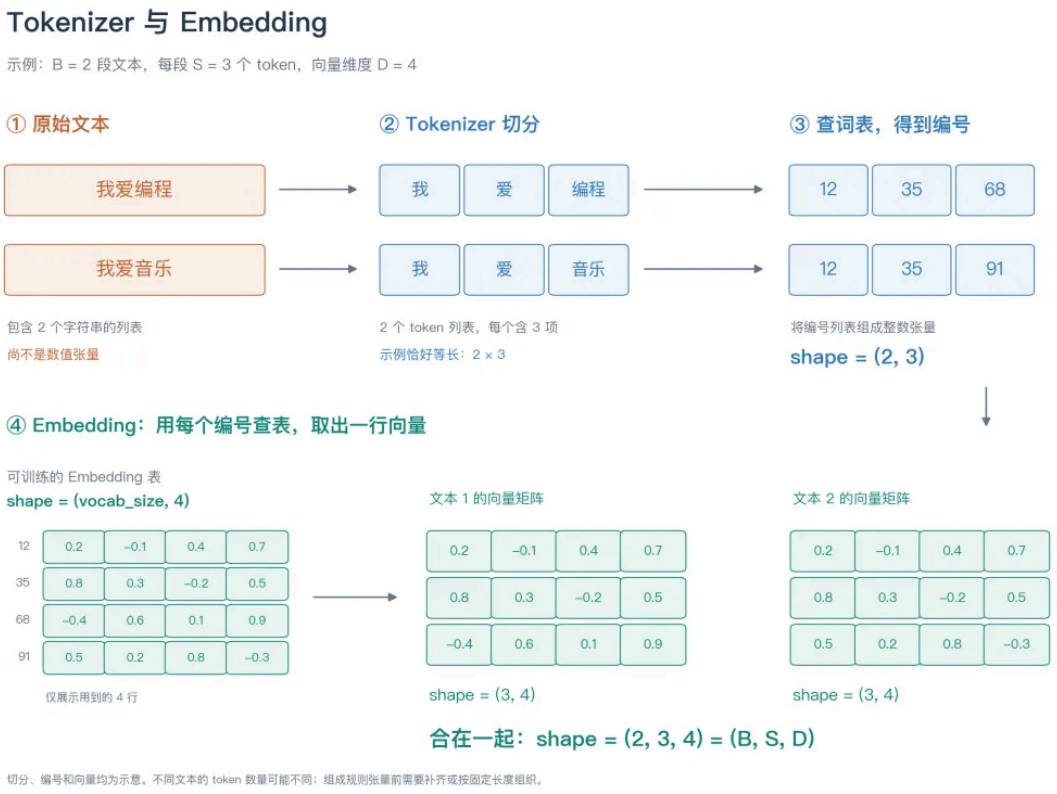
Embedding 把每个 token 编号转换成一个可学习的向量。 通过训练，「国王」和「皇后」这样含义相关的 token，可以在向量空间中更接近。这样，模型就能通过向量之间的关系来表示语义上的相似性，并将这些向量交给后面的 Attention 和前馈网络处理。

实现上，我们把所有 token 的向量放在一张表里。词表里有 `vocab_size` 个 token，每个向量有 `d_model` 个数，表的形状就是 `(vocab_size, d_model)`。
像 `vocab_size` 和 `d_model` 这种需要在创建模型时就确定的参数就叫超参数。

实际处理时，我们通常会把多段文本组成一批（batch），一起交给模型。比如一批有 2 段文本，每段有 3 个 token，编号就可以排成一个 2 行、3 列的数组，形状记作 `(2, 3)`。用 `B` 表示一批的文本条数、`S` 表示每段的 token 数量，就写成 `(B, S)`。

经过 Embedding，每个编号都被替换成一个向量。如果每个向量有 4 个数，输出形状就变成 `(2, 3, 4)`，表示 **2 段文本，每段 3 个 token，每个 token 用 4 个数表示**。一般写成 `(B, S, D)`，其中 `D` 就是向量的维度。

在拆解 Transformer 的过程中，我们要始终留意张量的 shape。看懂每个维度代表什么，以及数据经过各个模块后如何变化，就更容易理解模型内部的计算过程。



下面的公式表示，取出第 `b` 段文本中第 `s` 个 token 的编号，再去表 `E` 中查出对应向量。

$$x_{b,s} = E[\text{token_id}_{b,s}], \qquad E \in \mathbb{R}^{V \times D}$$

完整的 `Embedding` 实现如下。

```python
import torch
import math
class Embedding(torch.nn.Module):
    def __init__(self, vocab_size: int, d_model: int, device=None, dtype=None
        super().__init__()

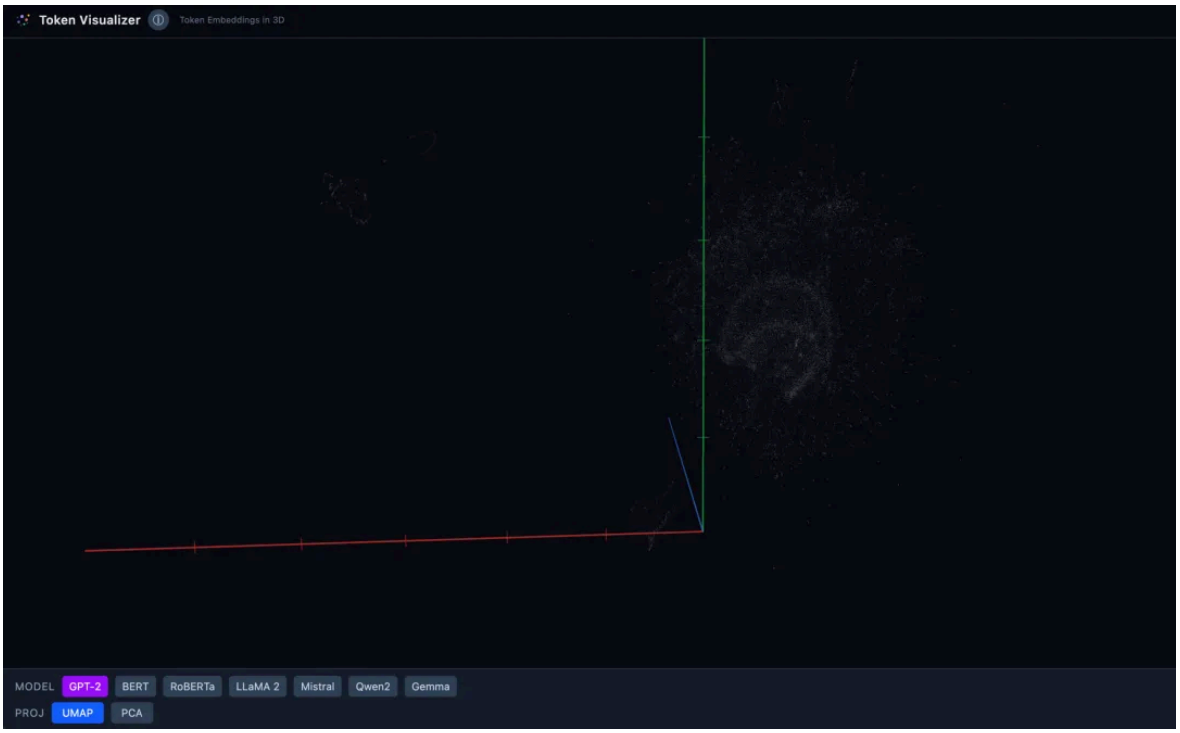
        # vocab_size 词表大小
        # d_model 每个 token 的维度
        self.embedding = torch.nn.Parameter(
            torch.empty(
                (vocab_size, d_model),
                device = device,
                dtype = dtype
            )
        )

        # 用 assignment1 里面给出的初始化公式进行初始化
        # W ~ 𝒩(μ = 0, σ² = 1, 截断到 [-3, 3]
        torch.nn.init.trunc_normal_(
            self.embedding,
            mean = 0,
            std = 1,
            a = -3,
            b = 3
        )

    # embedding 实际上就是一个查表操作
    # forward 的输入 tensor shape 为 (batch, seq_len)
    # 里面执行的逻辑就是对每个 batch 的每个 seq 元素，执行一下查表操作

    # 怎么理解这个 []？
    # 实际上 python 中 A[B] 等价于 A.__getitem__(B)
    def forward(self, token_ids: torch.Tensor) -> torch.Tensor:
        return self.embedding[token_ids]
```

关于 Embedding 同样也推荐一个网站：Token3D，它将模型的 Embedding 表中的每个 token 向量从高维度映射到低维度，可以直观的看到 Embedding 是什么样的，我们同样选择 gpt2，图中有 50257 个点：



> **扩展：Embedding 层与 Embedding 模型**
> 相信很多人之前在做语义搜索、RAG 时用过 Embedding 模型。它和这里的 Embedding 层都把输入转换成向量，但作用和处理过程不同。
> 这里的 **Embedding 层**是模型中的一个组件，按 token 编号查表，为每个 token 提供初始向量。同一个 token 出现在不同句子里，查到的向量相同，这一步还没有结合上下文。
> 常见的文本 **Embedding 模型**则接收一整段文字，经过包括 Attention 在内的多层计算，再汇总成一个表示整段文本的向量，用来比较文本的语义相似度。它内部通常也有 Embedding 层，但我们拿来做检索的向量，是整个模型处理后的结果。

5.3 Linear 与 SwiGLU

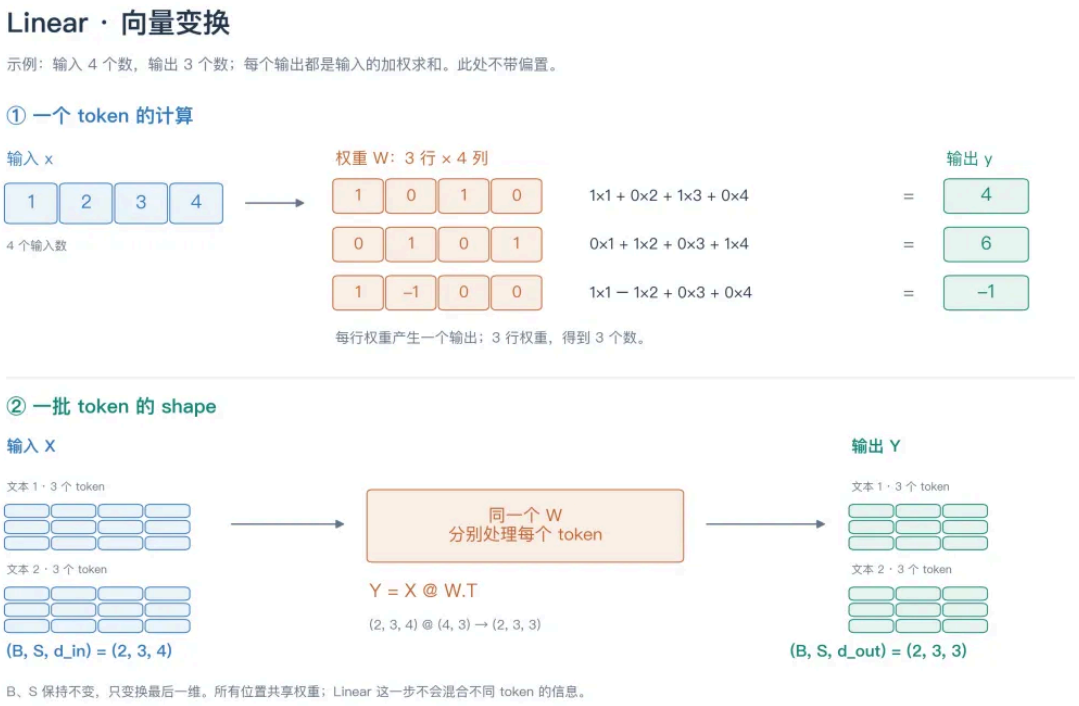
Linear

经过 Embedding，每个 token 已经有了一个向量。**Linear 做的就是把一个向量通过线性计算，转换成另一个向量。** 具体来说，它用一组可学习的权重，对输入的各个分量做加权求和，组成新的向量。后面 Attention 中的 Q、K、V，以及 SwiGLU 中的向量变换，都会用到它。

先看前面提到的线性回归的例子。那时一个输入对应一个权重，算的是 `y = wx + b`。现在输入有多个数，每个数都有自己的权重，把它们分别相乘再加起来，就得到一个输出。

按照 Assignment1 的要求，作业采用了 PaLM、LLaMA 等模型中省略线性层偏置的设计，因此我们实现的是 `y = Wx`，省略了偏置 `b`。

假设一个 token 的向量是 `[1, 2, 3, 4]`，我们希望输出 3 个数，就需要 3 组权重，每组都有 4 个数。把它们排在一起，得到一张 `3 × 4` 的权重矩阵 `W`。图中第一组权重是 `[1, 0, 1, 0]`，算出的第一个输出就是 `1×1 + 0×2 + 1×3 + 0×4 = 4`。另外两组分别算出 `6` 和 `-1`，于是输出向量变成 `[4, 6, -1]`。



这里的整数权重是为了方便看清计算过程。实际使用时，权重先随机初始化，再通过训练更新，模型会学到怎样组合输入更有利于预测。`in_features` 决定每组权重有多少个数，`out_features` 决定有多少组权重，也就决定了输出向量的长度。即使输入输出长度相同，Linear 也能改变向量的内容。

如果把单个输入向量写成一列，上面的计算就是：

$$y = Wx, \qquad W \in \mathbb{R}^{d_{out} \times d_{in}}$$

代码里，每个 token 的向量放在张量的最后一维，相当于横着排。因此计算写成 `X @ W.T`，`W.T` 就是对 W 进行转置操作。原本 `W` 的每一行是一组权重，转置后每一列是一组权重，刚好能与输入向量相乘求和。

$$Y = XW^{\mathsf{T}}$$

对于前面的 batch 示例，输入 shape 是 `(2, 3, 4)`，表示 2 段文本、每段 3 个 token、每个 token 有 4 个数。乘上 shape 为 `(4, 3)` 的 `W.T` 后，输出就变成 (2, 3, 3)。6 个 token 都分别使用同一个 W 做变换，文本条数和 token 数量保持不变，只有最后一维从 4 变成了 3。 这一步只处理每个 token 自身的向量，不会读取其他位置的 token。

下面是完整的 `Linear` 实现。按照 Assignment1 的要求，权重的初始化采用截断正态分布，均值为 0，标准差为 $\sqrt{2/(d_{in} + d_{out})}$，取值限制在正负三倍标准差内。

```
import
 torchimport mathclass Linear(torch.nn.Module):
```

```
def

__init__
(
self, in_features, out_features, device=
None
, dtype=
None
):
super
().__init__()
self
.in_features = in_features
self
.out_features = out_features
# 用 troch empty 申请出一个 tensor，devie、dtype 直接透传          # in_features
```

SwiGLU

前面 TinyGPT 的前馈模块用的是 `Linear → GELU → Linear` 这样的普通 MLP。Assignment1 里面要求的是另一种 MLP：**SwiGLU**。

它让输入走两条分支，一条计算特征，另一条计算一组系数，然后把对应位置的数相乘。比如某个特征的值是 `4`，乘上 `0.5` 就变成 `2`，乘上 `0` 就被抑制了。**这种用一条分支的输出调节另一条分支的计算方式，叫作门控。**

这些系数会随输入变化，让网络能灵活调节各个特征的强弱。GLU Variants Improve Transformer 的 T5 预训练实验表明，在相近的参数量和计算量下，SwiGLU 比使用 ReLU、GELU 的普通前馈层取得了更低的困惑度。

先看它用到的激活函数 SiLU：

$$\mathrm{SiLU}(x) = x \cdot \mathrm{sigmoid}(x) = \frac{x}{1 + e^{-x}}$$

下面这张作业原图比较了 SiLU 和 ReLU。SiLU 在 0 附近是平滑的，也允许一部分负值通过。

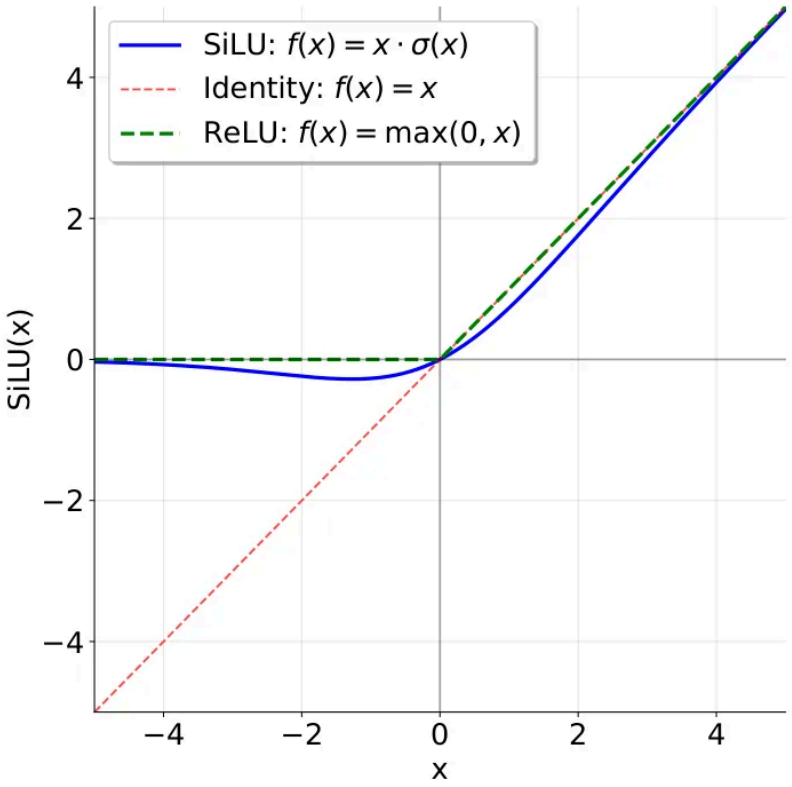
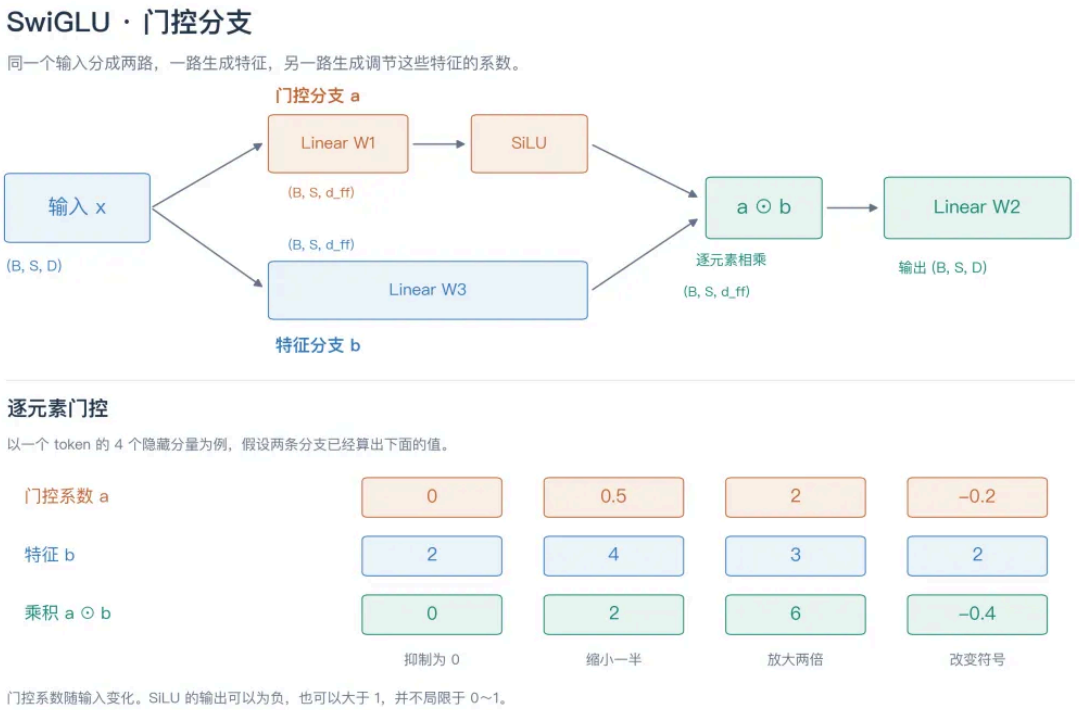


Figure 3: Comparing the SiLU (aka Swish) and ReLU activation functions.

SwiGLU 用到了三个不带偏置的 Linear，它们各自的权重矩阵记作 **W1、W2、W3**，都会随训练更新。输入 `x` 同时经过两个 Linear。权重为 `W3` 的一层生成特征；权重为 `W1` 的一层，其输出经过 SiLU 得到门控系数。两路结果逐元素相乘后，再经过权重为 `W2` 的 Linear 得到输出。



图中假设门控分支输出 `[0, 0.5, 2, -0.2]`，特征分支输出 `[2, 4, 3, 2]`。对应位置相乘后，得到 `[0, 2, 6, -0.4]`。第一个分量被抑制为 0，第二个缩小一半；后两个分别被放大和改变符号。SiLU 的输出可以为负，也可以大于 1，所以这里的门控能做连续的调节。

两路都先把向量从 `D` 扩到 `d_ff`，相乘后再通过 `W2` 变回 `D`。整个过程对每个 token 分别进行，batch 和序列长度保持不变。写成公式就是

$$\mathrm{SwiGLU}(x) = W_2(\mathrm{SiLU}(W_1 x) \odot W_3 x)$$

其中 $\odot$ 表示逐元素相乘。

作业建议 d_ff 取接近 $8D/3$ 的 64 的倍数。这样三个权重矩阵的参数量，接近传统隐藏宽度为 4D 的两层前馈网络。

完整实现包含 `silu()` 和 `SwiGLU`，三个投影都复用前面写好的 `Linear`。

```python
import torch
import math
def silu(in_features: torch.Tensor):
    return in_features * torch.sigmoid(in_features)
class SwiGLU(torch.nn.Module):
    def __init__(self, d_model: int, d_ff: int, device=None, dtype=None):
        # d_ff: 隐藏层的维度

        super().__init__()
        self.d_model = d_model
        self.d_ff = d_ff

        # w1、w3 负责升维
        self.w1 = Linear(d_model,d_ff,device,dtype)
        self.w3 = Linear(d_model,d_ff,device,dtype)

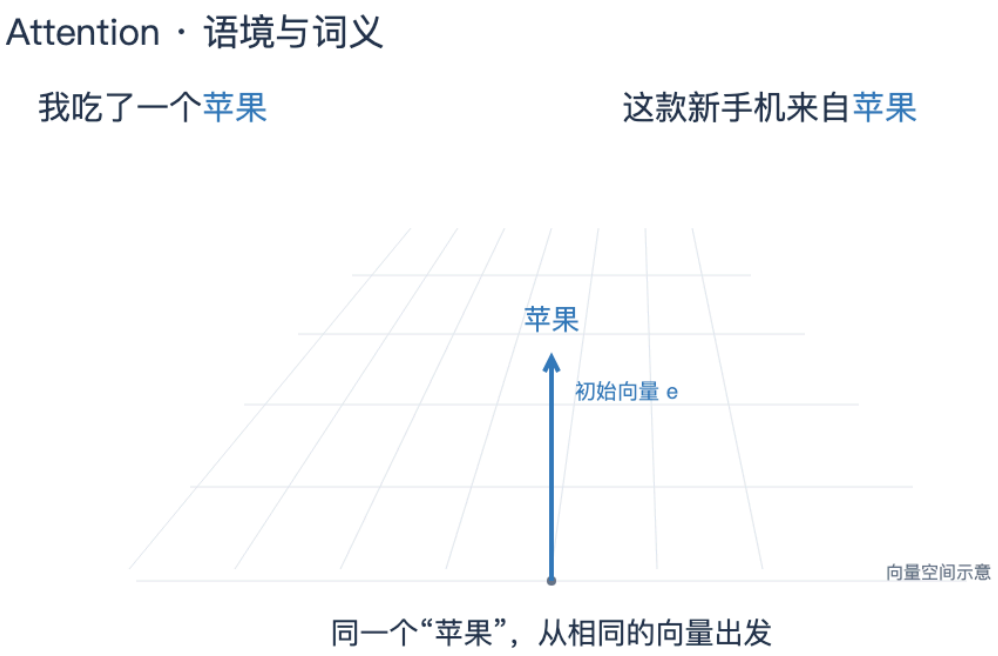
        # w2 负责降维
        self.w2 = Linear(d_ff,d_model,device,dtype)

    def forward(self, x: torch.Tensor):
        a = silu(self.w1(x))
        b = self.w3(x)
        # 注意这里是是逐元素相乘
        return self.w2(a * b)
```

5.4 Attention

“我吃了一个苹果”和“这款新手机来自苹果”，两句话里的“苹果”分别指水果和公司。假设“苹果”是一个 token，它在两句话中查到的 Embedding 向量相同，单靠这个向量无法区分当前说的是哪种含义，还需要结合“吃了”“新手机”等上下文。

前面的 Linear 和 SwiGLU 都分别处理每个 token 的向量。**Attention 让不同 token 的信息联系起来**：为可以读取的各个 token 计算权重，再按权重汇总它们的信息。这样，同一个 token 在不同语境中就可以得到不同的向量表示。



接下来就用“这款新手机来自苹果”拆开看。为了方便演示，假设它被切成 `这款` / `新` / `手机` / `来自` / `苹果` 五个 token。我们先看“苹果”怎样读取前文的信息，得到结合了当前语境的表示。

> 如果觉得这块内容比较抽象，推荐看一下 3Blue1Brown 的视频，动画演示会清晰很多：https://www.bilibili.com/video/BV1TZ421j7Ke

Q、K、V

在“这款新手机来自苹果”里，我们看到“手机”，就容易判断“苹果”指的是公司。我们希望模型也能利用这样的线索：处理“苹果”时，多关注能帮助理解它的词，把这些词的信息结合进来。

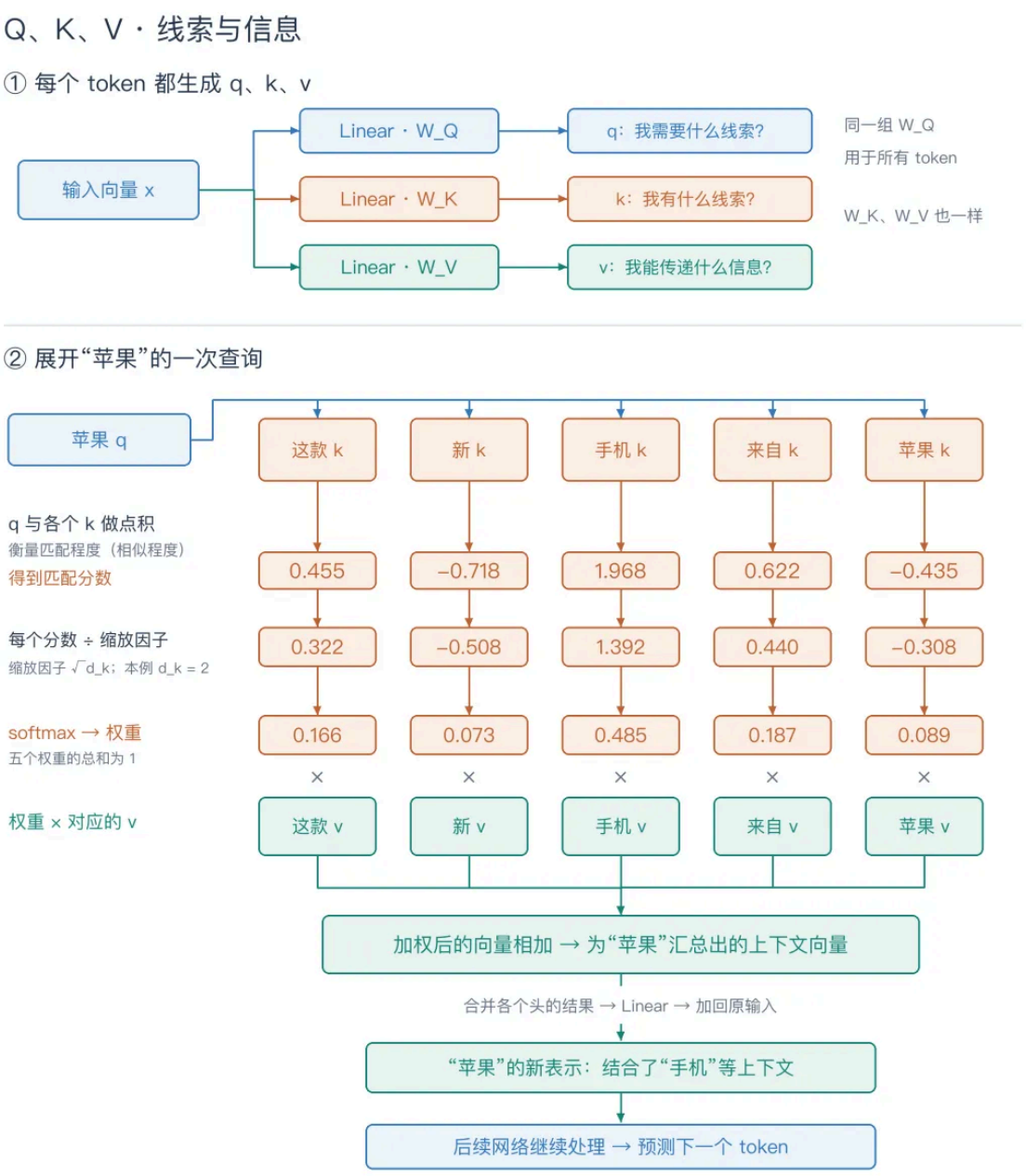
那模型怎么知道该关注谁，又该从对方那里拿到什么信息？Attention 为每个 token 算出三个向量，叫作 **q、k、v**。先用下面的说法理解它们：

向量	可以怎样理解	放到这个例子里
q（Query，查询）	我需要什么线索？	“苹果”需要能帮助判断当前含义的上下文线索
k（Key，键）	我有什么可供匹配的线索？	可以把“手机”的 k 想成带有“电子产品”这类标签，供“苹果”的 q 来匹配
v（Value，值）	我能传过去什么信息？	匹配后，“手机”通过 v 把自身的信息传给“苹果”

这些说法是帮助理解的比喻。q、k、v 实际上都是向量，由每个 token 的输入分别经过三个 Linear 算出来。

q 表示“想找什么”，k 表示“有什么可供匹配”。 两个向量做点积，可以把它们的匹配程度变成一个分数。比如“苹果”的 q 与“手机”的 k 匹配分数较高，经过缩放和 softmax 后，“手机”就会分到较大的权重。**点积得到的是分数，经过缩放和 softmax 才是权重。**

v 是实际传递的信息。 “手机”的权重越大，它的 v 对最终结果的贡献就越大。我们用 q、k 算“关注谁、关注多少”，用 v 提供“拿到什么信息”。



图中的缩放因子通常取 $\sqrt{d_k}$，其中 `d_k` 是 q、k 向量的分量个数。本例各有两个分量，所以缩放因子是 $\sqrt{2}$。向量越长，点积累加的项越多，分数的波动通常也越大；缩放可以避免 softmax 过早把权重集中到少数 token 上，让训练更稳定。

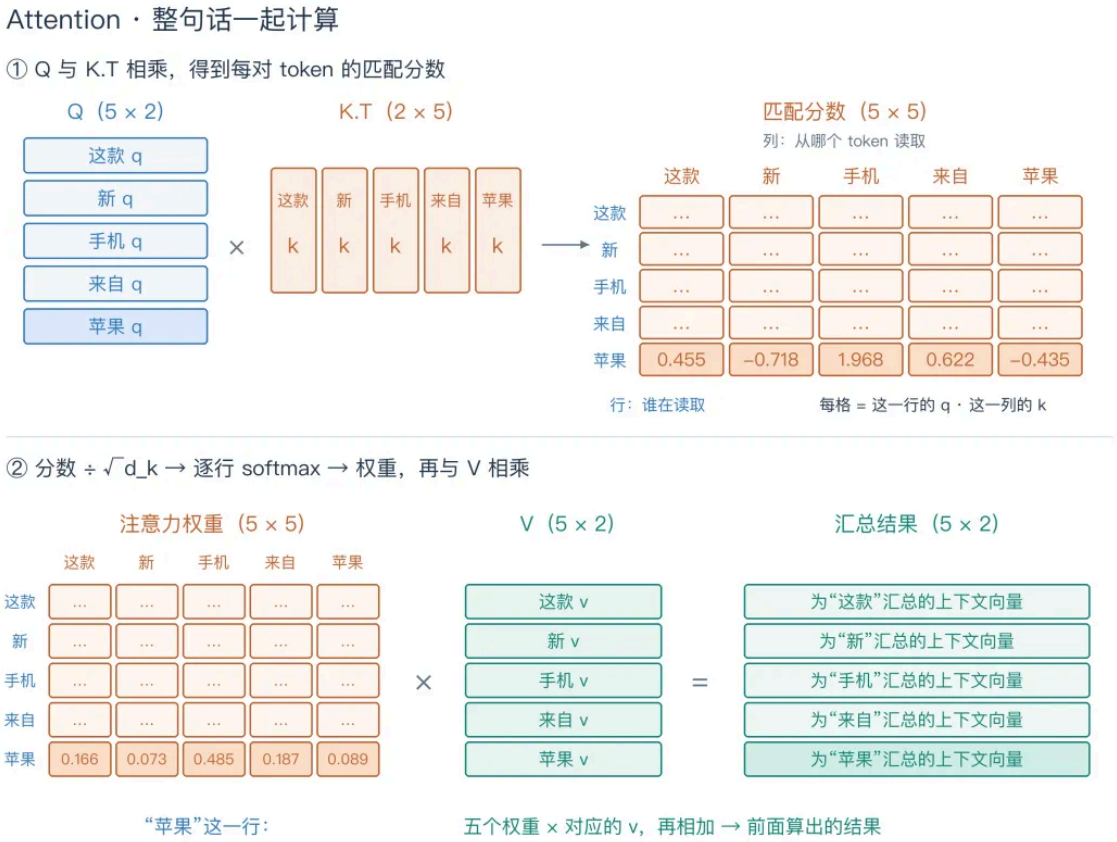
图中“手机”的权重约为 0.485，就把它的 v 中每个数乘以 0.485。其他 token 同样处理，再把五个向量相加，就得到一个汇总了上下文信息的向量，这就是这个注意力头的输出。

这个向量用来更新“苹果”原来的表示，让它带上“手机”等词提供的线索。后续网络就能利用这些线索理解当前语义，并最终预测下一个 token。图中也接上了这条后续路径，具体结构会在后面展开。

三个 Linear 的权重 `W_Q、W_K、W_V` 通过训练学习；q、k、v 和注意力权重则根据当前输入计算。比如 `q = x @ W_Q.T`，k、v 同理。

刚才只算了“苹果”。其他 token 也用自己的 q 做同样的计算。把所有 q 按行排成矩阵 Q，k、v 也分别排成 K、V，就能用矩阵乘法一起算。

沿用上面的示例，5 个 token 的 q、k、v 都用两个分量表示，因此 Q、K、V 的 shape 都是 `(5, 2)`。K 转置后，每一列就是一个 token 的 k。图中突出“苹果”这一行，其他行也按同样的方式计算。



分数表的每一行代表谁在读取信息，每一列代表从谁那里读取。 比如第 5 行就是“苹果”对五个 token 的匹配分数。这样，公式里的每一步就能和刚才的计算对应起来：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

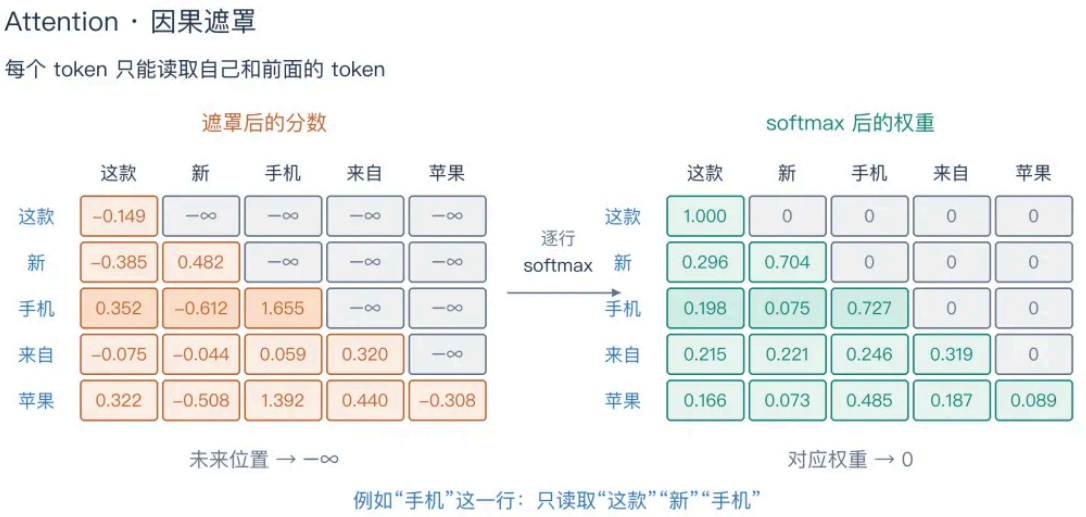
因果遮罩

模型生成文字时，只能根据已经出现的内容预测下一个 token。比如输入“这款 / 新”，接下来要预测“手机”，此时模型还看不到“手机 / 来自 / 苹果”。

训练时，我们已经有完整的句子。为了同时计算多个位置的预测，会把“这款 / 新 / 手机 / 来自 / 苹果”一起送进模型：“新”所在位置预测“手机”，“手机”所在位置预测“来自”。**如果 Attention 能读取后面的 token，“新”在预测“手机”时就能直接看到“手机”，相当于提前看到了答案。** 这样训练出来的模型就可能依赖生成时根本拿不到的信息。

因果遮罩就是把每个位置后面的 token 遮住。 这样既能让整句话并行训练，又能保证每个位置都只根据自己和前面的内容预测，与实际生成时的条件一致。

具体做法是：在 softmax **之前**，把未来位置的分数设为负无穷。经过 softmax，这些位置的权重就变成 0，其余权重在每行内归一化。

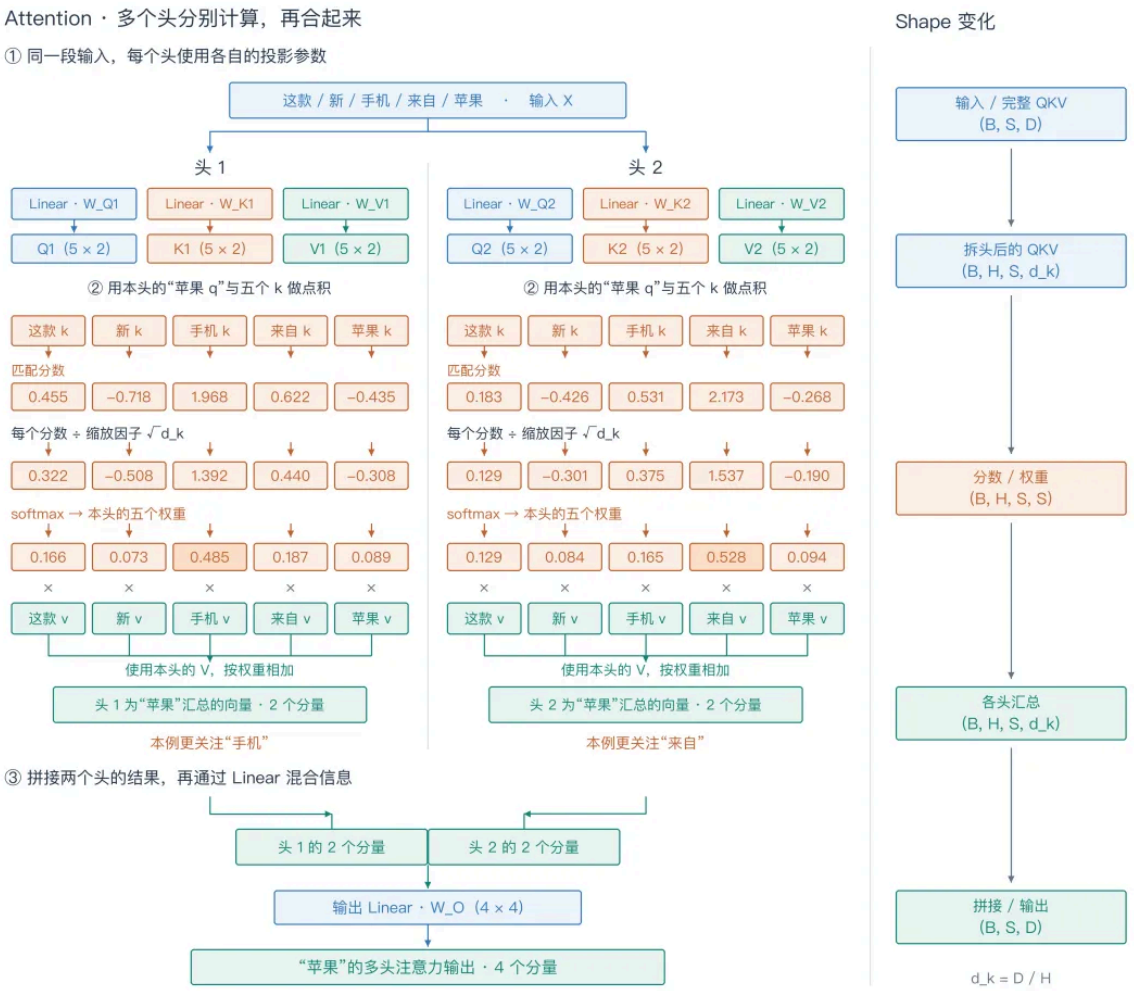


例如“手机”这一行，只能读取“这款”“新”“手机”，“来自”和“苹果”的权重为 0。最后一行的“苹果”可以读取全部五个 token，因此仍是前面算出的那组权重。

多头注意力

上面一个注意力头，为每个 token 算出了一组权重。**多头 Attention 让模型同时计算多组权重，分别汇总信息。** 各个头使用不同的投影参数，可以学到不同的匹配关系，最后把各自的结果合在一起。

比如处理“这款新手机来自苹果”中的“苹果”时，可以设想一个头更关注“手机”，提取电子产品相关的信息；另一个头更关注“来自”，提取来源关系的信息。两个头各自算权重、汇总向量，再把结果拼接起来，交给一个 Linear 混合。这样，“苹果”的表示就能同时结合不同的线索。这里的分工只是帮助理解，实际每个头关注什么由训练学到。



图里的 `B`、`S`、`D` 分别表示文本数量、序列长度和向量长度。拆成 `H` 个头后，每个头处理 `D / H` 个分量，独立计算一张 `S × S` 的注意力权重表；最后把各头的结果拼接，再经过输出 Linear，恢复成 `(B, S, D)`。

比如输入是 `(2, 5, 64)`，分成 4 个头后，Q、K、V 各自变成 `(2, 4, 5, 16)`。每个头汇总出 16 个分量，4 份拼回 64 个分量，最终输出仍是 `(2, 5, 64)`。

把图里的过程写成公式，就是：

$$
\begin{aligned}
Q = XW_Q^\mathsf{T}, \quad K = XW_K^\mathsf{T}, \quad V = XW_V^\mathsf{T} \\
\mathrm{head}_i = \mathrm{Attention}(Q_i, K_i, V_i) \\
\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_H) \\
\mathrm{MultiHeadSelfAttention}(X) = \mathrm{MultiHead}(Q, K, V) W_O^\mathsf{T}
\end{aligned}
$$

其中 Q_i, K_i, V_i 是分给第 i 个头的那部分向量；每个头都按前面介绍的方法计算 Attention，并使用因果遮罩。Concat 把各个头的输出沿最后一维拼接，W_O 对应图中最后的输出 Linear。

每段文本各自计算 Attention，batch 中不同文本之间不会互相读取。最后的 `out_proj` 沿用前面讲过的 Linear，对拼接后的向量进行变换。整个模块的输入、输出都是 `(B, S, D)`，但输出的每个 token 向量已经汇集了它能读取的上下文。

下面是 `softmax()`、注意力计算和多头 Attention 的完整代码，复用前面实现的 `Linear`。

```python
import torch
import math
def softmax(x: torch.Tensor, dim: int):
    # 因为 exp 是指数函数，如果 x 过大可能会导致溢出，而 softmax 实际上只关心每个 x
    x = x - x.max(dim = dim, keepdim = True).values
    exp_x = torch.exp(x)
    return exp_x / exp_x.sum(dim = dim,keepdim = True)
def scaled_dot_product_attention(
    Q: torch.Tensor,
    K: torch.Tensor,
    V: torch.Tensor,
    mask: torch.Tensor | None = None,
) -> torch.Tensor:
    """
    参数：
        Q: [..., n, d_k]    （n 为 Query 序列长度）
        K: [..., m, d_k]    （m 为 Key 序列长度）
        V: [..., m, d_v]
        mask: [n, m] 布尔矩阵，True 为保留，False 为屏蔽
    返回：
        [..., n, d_v] 与输入 batch 维一致的张量
    """
    # 先获取 K 向量维度
    d_k = K.size(-1)

    # 计算 Q K 向量的分数，并除以 √d_k 进行缩放
    socres = (Q @ K.mT) / math.sqrt(d_k)

    """
    Masked Scores = [[10, -inf, -inf],
                     [ 5,  10, -inf],
                     [ 2,   8,  10]]
        以下权重只示意遮罩后的形状，并非上面分数的实际 softmax 结果：
        [[1.0, 0.0, 0.0],
                       [0.3, 0.7, 0.0],
                       [0.1, 0.4, 0.5]]
    """
    if mask is not None:
        # mask 为 flase的地方设为负无穷
        socres = socres.masked_fill(mask == False, float('-inf'))

    socres = softmax(socres, dim = -1)

    return socres @ V
class MultiheadSelfAttention(torch.nn.Module):
    def __init__(self, d_model: int, num_heads: int, device=None, dtype=None)
```

```
        super().__init__()

        # 要用 head 数量均分 d_model，所以需要校验能否整除
        assert d_model % num_heads == 0

        self.d_model = d_model
        self.num_heads = num_heads
        self.head_dim = d_model // num_heads

        # 先定义获取 QKV 向量的 Linear
        self.W_Q = Linear(in_features=d_model, out_features=d_model, device=d
        self.W_K = Linear(in_features=d_model, out_features=d_model, device=d
        self.W_V = Linear(in_features=d_model, out_features=d_model, device=d

        # 输出经过 QKV 运算后需要再过一次 Linear
        self.out_proj = Linear(in_features=d_model, out_features=d_model, dev

    def forward(self, x: torch.Tensor):
        # 先计算出完整的 QKV，再按照 head 拆分
        q = self.W_Q(x)
        k = self.W_K(x)
        v = self.W_V(x)

        # 拆分后的多头 QKV 的 shape：(batch_size, num_heads, seq_len, head_dim)
        # 这里 view 拆分需要先拆 qkv 的最后维度，所以前两维 shape 要展示保持 batch
        batch_size, seq_len, d_model = x.shape
        q = q.view(batch_size, seq_len, self.num_heads, self.head_dim).transp
        k = k.view(batch_size, seq_len, self.num_heads, self.head_dim).transp
        v = v.view(batch_size, seq_len, self.num_heads, self.head_dim).transp

        # 这里需要一个下三角矩阵
        """
        True  False False False
        True  True  False False
        True  True  True  False
        True  True  True  True
        """
        mask = torch.tril(torch.ones(seq_len, seq_len, device=x.device, dtype

        # 计算注意力
        out = scaled_dot_product_attention(Q=q,K=k,V=v,mask=mask)

        # 把 heads 维度拼回 d_model，恢复 (batch_size, seq_len, d_model)。
        out = out.transpose(1, 2).contiguous().view(batch_size, seq_len, self
        return self.out_proj(out)
```

5.5 RoPE

上一节的基础 Attention 根据 q、k 的匹配程度分配权重。因果遮罩限制了读取范围；我们还希望匹配分数能利用 token 之间相隔多远的信息。

TinyGPT 的做法是把位置 Embedding 加到 token Embedding 上，再计算 q、k、v。这里改用 **RoPE（旋转位置编码）**，在 q、k 已经计算出来之后，根据各自的 token 位置旋转它们，再计算注意力分数。

RoPE 的具体做法是：把 q、k 的高维向量按分量两两配对，每一对当成一个二维向量，根据 token 的位置分别旋转，再按原顺序拼回去。 比如 (a, b, c, d) 拆成 (a, b) 和 (c, d)，旋转后得到 (a', b') 和 (c', d')，拼起来仍然是一个四维向量。 要理解这种做法为什么成立，我们先来复习一下高中就学过的二维旋转矩阵：

$$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}, \qquad \begin{pmatrix} a' \\ b' \end{pmatrix} = R(\theta) \begin{pmatrix} a \\ b \end{pmatrix}$$

它把二维向量逆时针旋转 θ，长度保持不变。比如 $(1, 0)$ 旋转 90°，就变成 $(0, 1)$。 回到刚才的四维例子，对 (a, b) 和 (c, d) 分别旋转，等价于让整个向量乘下面这个矩阵：

$$\begin{pmatrix} a' \\ b' \\ c' \\ d' \end{pmatrix} = \begin{pmatrix} \cos\alpha & -\sin\alpha & 0 & 0 \\ \sin\alpha & \cos\alpha & 0 & 0 \\ 0 & 0 & \cos\beta & -\sin\beta \\ 0 & 0 & \sin\beta & \cos\beta \end{pmatrix} \begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix}$$

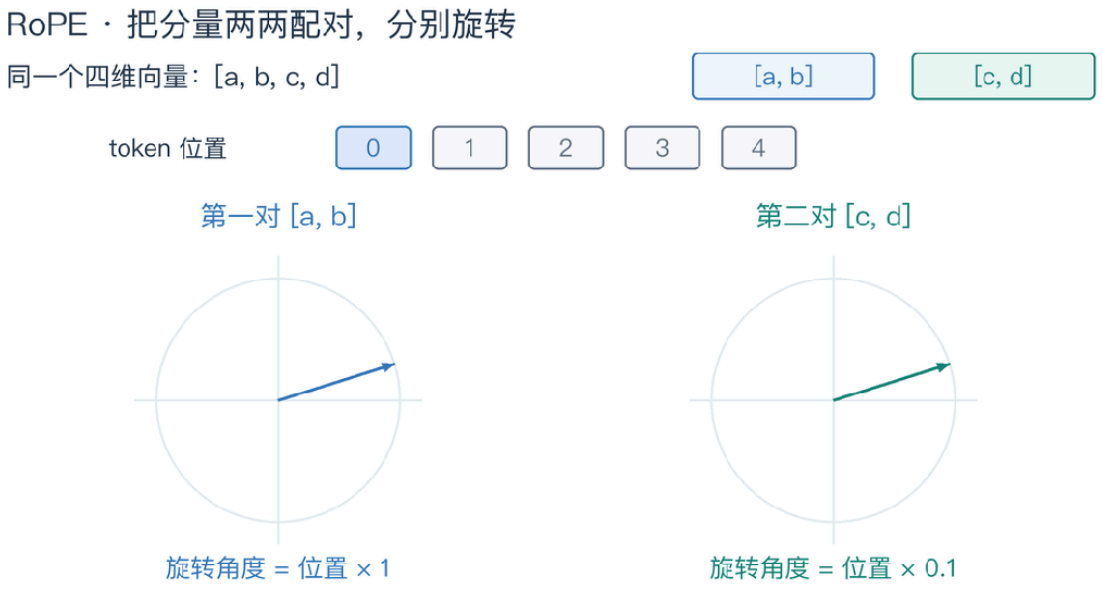
左上角只旋转 (a, b)，右下角只旋转 (c, d)，这种结构叫分块对角矩阵。每一对旋转后长度不变，所以整个向量的长度也不变；知道旋转角度后，反向旋转就能还原，信息没有被丢掉。

更高维的向量也可以这样处理，只需沿对角线放入更多二维旋转矩阵。因此这里要求向量长度为偶数。配对按固定的坐标顺序进行；计算 q、k 的点积时，各对分量的点积相加，就得到完整向量的点积。

接下来确定每一对转多少。这里 `i` 表示 token 在序列中的位置，从 0 开始；`k` 表示向量中第几对分量，从 1 开始。比如四维向量 `(a,b,c,d)` 中，`k=1` 对应 `(a,b)`，`k=2` 对应 `(c,d)`。位置 `i` 的第 `k` 对分量，旋转角度为：

$$\theta_{i,k} = \frac{i}{\Theta^{(2k-2)/d_k}}, \qquad k = 1, \ldots, d_k/2$$

Θ 是控制旋转频率的基数。比如取 $d_k = 4$、$\Theta = 100$，向量拆成两对后，第一对每前进一个位置旋转 1 弧度，第二对旋转 0.1 弧度。下面固定向量的内容，看看它放在不同位置时会怎样旋转：

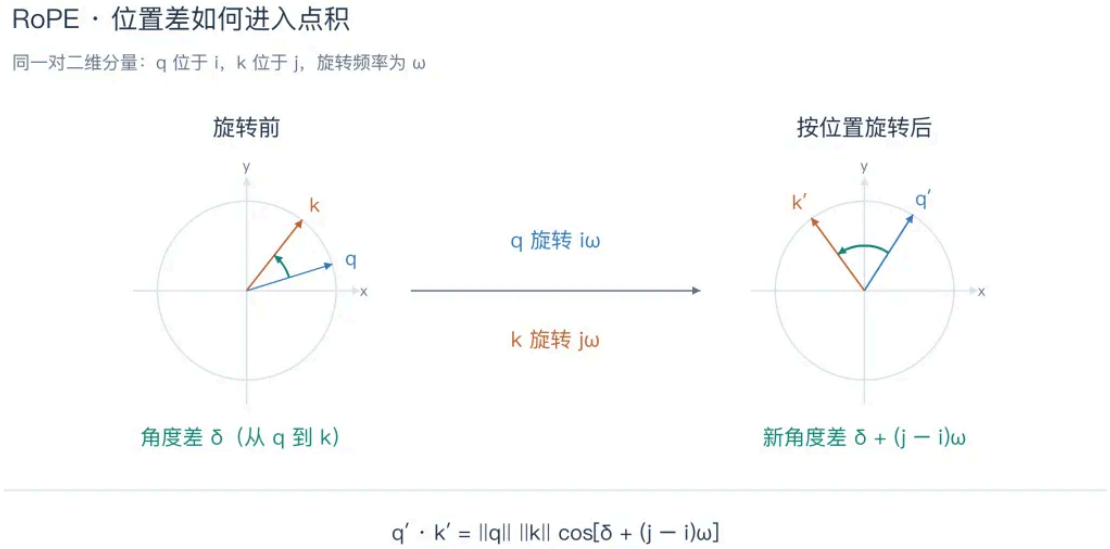


位置 0：两对分量使用不同的旋转频率

我们可以把不同分量对想成钟表上的秒针和时针：**秒针转得快，方便看几秒钟的变化；时针转得慢，方便看几小时的变化。** RoPE 给不同分量对设置不同频率，也是为了提供不同距离尺度的位置线索。

- **高频分量像秒针，对较近的位置变化更敏感。** 上面第一对每前进一个 token 就转 1 弧度，约 57°，相邻位置的方向差别已经很明显。但它转得快，绕过一圈后又会接近原来的方向，单看这一对，较远的位置也可能看起来很接近。
- **低频分量像时针，在更长的位置范围内缓慢变化。** 第二对每前进一个 token 只转 0.1 弧度，约 5.7°，前进 10 个位置才转 1 弧度。它转一圈需要跨过更长的距离，能补充较远距离的位置线索；但如果只用很慢的频率，相邻位置的角度差又会太小。

所以我们把快慢不同的“指针”放在一起：快的帮助区分附近的位置，慢的补充更长范围的变化。这些分量具体承载什么内容，由模型训练学到。



$$q' \cdot k' = \|q\| \, \|k\| \cos[\delta + (j - i)\omega]$$

点积等于两个向量的长度相乘，再乘以夹角的余弦。 只看图中这一对二维分量：原来从 q 到 k 的角度差是 δ，按位置分别旋转 $i\omega$、$j\omega$ 后，角度差就变成 $\delta + (j-i)\omega$：

$$q' \cdot k' = \|q\| \, \|k\| \cos\big(\delta + (j-i)\omega\big)$$

这里 ω 是这对分量的旋转频率。向量长度不变，**位置差 $j-i$ 通过角度差影响点积分数。**完整向量的点积，就是把各对分量的结果相加。

旋转保持 q、k 的模长和 shape 不变，v 不旋转。 后续仍按原来的方式计算注意力权重，再对 v 加权求和。相比把位置向量加到 token Embedding 上，RoPE 的优势是让匹配分数直接依赖相对距离。

下面是完整的 RoPE 以及集成了 RoPE 的多头注意力的实现：

```python
import torch
import math
class RotaryPositionalEmbedding(torch.nn.Module):
    def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None
        # theta：RoPE 的频率基数
        # d_k：q、k 向量维度
        # max_seq_len：最大 token 长度
        super().__init__()
        self.d_k = d_k

        # 先计算每个位置需要旋转的角度
        # 公式：f_k = 1 / theta^((2k - 2) / d_k)
        # arange(0, d_k, 2) 产生 [0, 2, 4, ..., d_k-2]，对应公式中的2k-2(k从1开
        # 注意这里 f_k 的 shape 是 (d_k / 2,)
        f_k = 1 / (theta ** (torch.arange(0, d_k, 2, device=device).float()
        # 这里要用外积，外积的结果是前行后列，angle 是一个矩阵
        # angle[n, k] = n × f_k = n / theta^((2k - 2) / d_k)
        # angle 的shape：(max_seq_len, d_k / 2)
        angle = torch.outer(torch.arange(max_seq_len, device=device).float()

        # cos_table、sin_table shape 都是 (max_seq_len, d_k / 2)
        self.register_buffer("cos_table", angle.cos())
        self.register_buffer("sin_table", angle.sin())

    def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> tor
        # x：shape 为 (..., seq_len, d_k) 的 q 或 k 向量
        # token_positions：shape 为 (..., seq_len) 的整数位置下标，每个位置对应

        # 这里 cos 和 sin 的 shape 都是 token_positions.shape + (d_k // 2,)，
        cos_value = self.cos_table[token_positions].repeat_interleave(2, dim
        sin_value = self.sin_table[token_positions].repeat_interleave(2, dim
```

```
        # 公式：rotated_q = q ⊙ cos_value + rotate_half(q) ⊙ sin_value
        # rotate_half：[x0, x1, x2, x3, ...] → [-x1, x0, -x3, x2, ...]

        """
        最后一维两两拆分，例如：
        原来 (3, 6)：
        [
            [ 1,  2,  3,  4,  5,  6],
            [ 7,  8,  9, 10, 11, 12],
            [13, 14, 15, 16, 17, 18],
        ]
        现在 (3, 3, 2)：
        [
            [ [1, 2],  [3, 4],  [5, 6] ],      ← 第 0 个 token，3 组，每组 2
            [ [7, 8],  [9,10],  [11,12] ],     ← 第 1 个 token
            [ [13,14], [15,16], [17,18] ],     ← 第 2 个 token
        ]
        """
        x_unflatten = x.unflatten(-1,(-1,2))

        """
        按照奇偶拆分开
        even = 每组的第 0 个元素：
        [
            [ 1,  3,  5],
            [ 7,  9, 11],
            [13, 15, 17],
        ]
        odd = 每组的第 1 个元素：
        [
            [ 2,  4,  6],
            [ 8, 10, 12],
            [14, 16, 18],
        ]
        """
        even, odd = x_unflatten.unbind(-1)

        """
        先奇偶互换，然后压缩最后两维
        stack 后的结果：
        [
            [ [-2,  1], [-4,  3], [ -6,  5] ],     ← 第 0 个 token
            [ [-8,  7], [-10, 9], [-12, 11] ],     ← 第 1 个 token
            [ [-14,13], [-16,15], [-18, 17] ],     ← 第 2 个 token
        ]
```

```
        flatten 后：
        [
            [ -2,  1,  -4,   3,  -6,   5],
            [ -8,  7, -10,   9, -12,  11],
            [-14, 13, -16,  15, -18,  17],
        ]

        """
        x_rotate_half = torch.stack((-odd, even), dim = -1).flatten(-2)

        rotated_x = x * cos_value + x_rotate_half * sin_value

        return rotated_x

class MultiheadSelfAttention(torch.nn.Module):
    def __init__(self, d_model: int, num_heads: int, theta=None, max_seq_len
        super().__init__()

        # 要用 head 数量均分 d_model，所以需要校验能否整除
        assert d_model % num_heads == 0

        self.d_model = d_model
        self.num_heads = num_heads
        self.head_dim = d_model // num_heads

        # 先定义获取 QKV 向量的 Linear
        self.W_Q = Linear(in_features=d_model, out_features=d_model, device=
        self.W_K = Linear(in_features=d_model, out_features=d_model, device=
        self.W_V = Linear(in_features=d_model, out_features=d_model, device=

        # 输出经过 QKV 运算后需要再过一次 Linear
        self.out_proj = Linear(in_features=d_model, out_features=d_model, de

        # 如果传入了 theta 和 max_seq_len，则处理 rope 的逻辑
        if theta is not None and max_seq_len is not None:
            self.rope = RotaryPositionalEmbedding(theta=theta, d_k=self.head
        else:
            self.rope = None

    def forward(self, x: torch.Tensor, token_positions=None):
        # 先计算出完整的 QKV，再按照 head 拆分
        q = self.W_Q(x)
        k = self.W_K(x)
        v = self.W_V(x)

        # 拆分后的多头 QKV 的 shape：(batch_size, num_heads, seq_len, head_di
```

```
        # 这里 view 拆分需要先拆 qkv 的最后维度，所以前两维 shape 要展示保持 batc
        batch_size, seq_len, d_model = x.shape
        q = q.view(batch_size, seq_len, self.num_heads, self.head_dim).trans
        k = k.view(batch_size, seq_len, self.num_heads, self.head_dim).trans
        v = v.view(batch_size, seq_len, self.num_heads, self.head_dim).trans

        # 如果 rope 不为 null 就对 q、k 进行 rope 计算
        if self.rope is not None:
            # token_positions shape: (batch_size, seq_len)
            if token_positions is None:
                token_positions = torch.arange(seq_len, device=x.device).exp
            # 每条文本的位置沿 head 轴广播；一维位置可由所有文本共用。
            if token_positions.ndim == 2:
                token_positions = token_positions.unsqueeze(1)
            q=self.rope(q, token_positions)
            k=self.rope(k, token_positions)

        # 这里需要一个下三角矩阵
        """
        True  False False False
        True  True  False False
        True  True  True  False
        True  True  True  True
        """
        mask = torch.tril(torch.ones(seq_len, seq_len, device=x.device, dtyp

        # 计算注意力
        out = scaled_dot_product_attention(Q=q,K=k,V=v,mask=mask)

        # 把 heads 维度拼回 d_model，恢复 (batch_size, seq_len, d_model)。
        out = out.transpose(1, 2).contiguous().view(batch_size, seq_len, sel
        return self.out_proj(out)
```

5.6 RMSNorm

向量经过多层计算，数值的大小会变化。我们希望送入下一步计算时，它们的整体尺度比较稳定。**RMSNorm 就像调节音量：先根据向量自身的大小做一次缩放，再让模型学习各个分量需要多大的幅度。**

每个 token 都有自己的 RMS。 一个 token 的向量里有 D 个数，就用这 D 个数计算它的 RMS，再用这个 RMS 归一化该向量。不同 token 分别计算，可以并行完成。

先算 RMS（Root Mean Square，均方根）。 设这个向量为 $x = (x_1, \ldots, x_D)$，把所有分量分别平方、求平均，再开根号，就得到衡量整体大小的一个数。为避免后面除以零，根号内还

$$\epsilon$$ $$\mathrm{RMS}(x) = \sqrt{\frac{1}{D}\sum_{j=1}^{D} x_j^2 + \epsilon}$$

加上小常数 D 是分量个数，j 遍历这些分量；ϵ这里取 10^{-5}，避免下一步除以零。 **再做归一化和缩放。** 每个分量 x_i 都除以上面算出的同一个 RMS，再乘自己的缩放系数 g_i：

$$\mathrm{RMSNorm}(x)_i = \frac{x_i}{\mathrm{RMS}(x)}\, g_i$$

i 表示当前分量。g_i 初始化为 1，随后通过训练学习，让模型调整这个分量的幅度。同一层的所有 token 共用这组 g，输入输出形状不变。 举个例子，令 $x = (2, 4)$、$g = (1, 1)$。忽略很小的 ϵ：

$$\mathrm{RMS}(x) = \sqrt{\frac{2^2 + 4^2}{2}} = \sqrt{10} \approx 3.162$$

于是输出约为 $(2/3.162,\ 4/3.162) = (0.632,\ 1.265)$。如果输入整体放大成 $(20, 40)$，RMS 也会放大十倍，归一化后仍得到相同的结果。

输入整体放大成 (20,40)，RMS 也会放大十倍，归一化后仍得到相同的结果。

和 LayerNorm 相比，RMSNorm 省去了减均值这一步。 LayerNorm 先减均值，再除以标准差；RMSNorm 直接除以均方根，计算步骤更少，也不要求归一化后的均值为 0。

下面是完整的 `RMSNorm` 实现。

```python
import torch
import math
class RMSNorm(torch.nn.Module):
    def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=No
        super().__init__()

        # d_model 模型 token 维度；weight 用于最后做缩放，维度要和 d_model 一致
        # eps 用于防止分母为 0

        self.weight = torch.nn.Parameter(
            torch.ones(
                d_model,
                device = device,
                dtype = dtype
            )
        )
        self.eps = eps
        pass
    def forward(self, x: torch.Tensor) -> torch.Tensor:
```

```
        # 先转成 float32，降低平方时溢出的风险
        in_dtype = x.dtype
        x_float32 = x.to(torch.float32)

        # 应用公式
        # 注意 mean 只需要对最后维度求平均，keepdim 表示需要保留维度，这里 mean 后
        rms = torch.sqrt(
            torch.mean(
                torch.square(x_float32),
                dim = -1,
                keepdim = True
            ) + self.eps
        )
        # 注意这里是 * weight 而不是 @，因为是逐元素相乘缩放
        # 这里计算需要注意一下 shape
        # x_floagt32(batch, seq_len, d_model)，rms(batch, seq_len, 1)
        # x_floagt32/rms 这里会用到 pytorch 里面的广播机制
        # 广播：两个 shape 不完全一样的 tensor 做逐元素运算时，PyTorch 会在尺寸为
        result = (x_float32 / rms) * self.weight

        # 返回前转回原来的 dtype
        return result.to(in_dtype)
```

5.7 组装 Transformer

前面的模块已经齐了，现在回过头来看 Architecture 中的那两张图，我们接下来只要按照架构去组装`TransformerBlock` 和 `TransformerLM` 就可以了：

```
import torch
import math
class TransformerBlock(torch.nn.Module):
    def __init__(self, d_model: int, num_heads: int, d_ff: int, theta=None, m
        super().__init__()

        # 初始化 MHA
        self.attention = MultiheadSelfAttention(
            d_model=d_model,
            num_heads=num_heads,
            theta=theta,
            max_seq_len=max_seq_len,
            device=device,
            dtype=dtype
        )

        # 初始化 FF
```

```
        self.ffn = SwiGLU(d_model=d_model, d_ff=d_ff, device=device, dtype=dt

        # 初始化两个 RMSNorm，分别用于 MHA、FF
        self.norm1 = RMSNorm(d_model=d_model, device=device, dtype=dtype)
        self.norm2 = RMSNorm(d_model=d_model, device=device, dtype=dtype)

    def forward(self, x: torch.Tensor, token_positions: torch.Tensor = None):
        # 先进行 MHA 操作
        x = x + self.attention(x=self.norm1(x), token_positions=token_positio
        # 再进行 FF 操作
        x = x + self.ffn(x=self.norm2(x))
        return x
class TransformerLM(torch.nn.Module):
    def __init__(self, vocab_size: int, nums_layer: int, d_model: int, num_he
                 theta=None, max_seq_len=None, device=None, dtype=None):

        super().__init__()

        self.token_embedding = Embedding(vocab_size=vocab_size, d_model=d_mod
        self.layers = torch.nn.ModuleList([
            TransformerBlock(
                d_model=d_model,
                num_heads=num_heads,
                d_ff=d_ff,
                theta=theta,
                max_seq_len=max_seq_len,
                device=device,
                dtype=dtype
            )
            for _ in range(nums_layer)
        ])

        self.norm = RMSNorm(d_model=d_model,device=device,dtype=dtype)

        # 注意最后是把向量映射回词表
        self.linear = Linear(in_features=d_model, out_features=vocab_size, de

    def forward(self, token_ids: torch.Tensor):
        batch_size, seq_len = token_ids.shape

        # 计算后 x.shape: (batch, size, d_model)
        x = self.token_embedding(token_ids)

        # 需要构建一个 token positions，传递到 transformer block 里面给 rope 使
        # 比如
        # batch_size = 2
```

```
        # seq_len = 4
        # [
        #  [0, 1, 2, 3],
        #  [0, 1, 2, 3],
        # ]
        token_positions = torch.arange(seq_len, device=token_ids.device).unsc

        # 计算后 x.shape: (batch, size, d_model)
        for layer in self.layers:
            x = layer(x, token_positions)

        # 计算后 x.shape: (batch, size, d_model)
        x = self.norm(x)

        # 这里要把向量通过 lm header 计算为词表分数
        # 计算后 x.shape: (batch, size, vocab_size)
        x = self.linear(x)

        return x
```

`TransformerBlock` 的 `forward()`:

$$y = x + \mathrm{MultiHeadSelfAttention}(\mathrm{RMSNorm}_1(x))$$
$$z = y + \mathrm{SwiGLU}(\mathrm{RMSNorm}_2(y))$$

x 是输入，y 是中间结果，z 是输出。这里的 + 就是残差连接，它把子层的计算结果加回输入，在原有表示上做更新。它也为梯度提供了一条直接传回前面各层的路径，让深层网络更容易训练。 TransformerLM 的 forward():

$$x^{(0)} = \mathrm{Embedding}(\text{token IDs})$$
$$x^{(\ell)} = \mathrm{Block}_\ell(x^{(\ell-1)}), \quad \ell = 1, \ldots, L$$
$$\text{logits} = \mathrm{LMHead}(\mathrm{RMSNorm}(x^{(L)}))$$

L 是 Block 的数量，$x^{(\ell)}$ 是第 ℓ 个 Block 的输出。LM head 将每个 token 的向量映射为词表分数 logits，尚未经过 softmax。

参数量和计算量

最后来看一个 Assignment1 里的资源核算题，把前面各个模块的参数量和计算量串起来:

Problem (`transformer_accounting`): Transformer LM resource accounting (5 points)

(a) Consider a GPT-2 XL-sized model using our assignment architecture, which has the following configuration:

- **vocab_size:**
  50,257
- **context_length:**
  1,024
- **num_layers:**
  48
- **d_model:**
  1,600
- **num_heads:**
  25
- **d_ff:**
  4,288 (the nearest multiple of 64 to $\frac{8}{3} \times 1,600$)

Suppose we constructed our model using this configuration. How many trainable parameters would our model have? Assuming each parameter is represented using single-precision floating point, how much memory is required to just load this model?

Deliverable: A one-to-two sentence response.

(b) Identify the matrix multiplies required to complete a forward pass of our GPT-2 XL-shaped model. How many FLOPs do these matrix multiplies require in total? Assume that our input sequence has `context_length` tokens.

Deliverable: A list of matrix multiplies (with descriptions), and the total number of FLOPs required.

(c) Based on your analysis above, which parts of the model require the most FLOPs?

Deliverable: A one-to-two sentence response.

(d) Repeat your analysis with GPT-2 small (12 layers, 768 d_model, 12 heads), GPT-2 medium (24 layers, 1024 d_model, 16 heads), and GPT-2 large (36 layers, 1280 d_model, 20 heads). As the model size increases, which parts of the Transformer LM take up proportionally more or less of the total FLOPs?

Deliverable: For each model, provide a breakdown of model components and its associated FLOPs (as a proportion of the total FLOPs required for a forward pass). In addition, provide a one-to-two sentence description of how varying the model size changes the proportional FLOPs of each component.

(e) Take GPT-2 XL and increase the context length to 16,384. How does the total FLOPs for one forward pass change? How does the relative contribution of FLOPs of the model components change?

Deliverable: A one-to-two sentence response.

我们就以 GPT-2 XL 的规模来计算一下这道题：

1. 参数量和内存： 按给定配置，模型有多少可训练参数？用 float32 存储这些参数需要多少内存？

我们把各个模块的参数加起来。这里所有 Linear 都不带偏置，Embedding 和 LM head 不共享权重：

○ Embedding 和 LM head 各有一张权重表，共 $2 \times 50257 \times 1600$ 个参数。

○ 每个 Block 中，Attention 的 Q、K、V 和输出投影共 4×1600^2 个参数；SwiGLU 的三个 Linear 共 $3 \times 1600 \times 4288$ 个参数；两个 RMSNorm 共 2×1600 个参数。

○ 最后一个 RMSNorm 还有 1600 个参数，RoPE 没有可训练参数。

把 48 个 Block 加上输入、输出部分，总参数量就是：

$$N = 2 \times 50257 \times 1600 + 48 \times (4 \times 1600^2 + 3 \times 1600 \times 4288 + 2 \times 1600) + 1600 = 1{,}640{,}452{,}800$$

约 16.40 亿参数。float32 每个参数占 4 字节，所以仅存储参数需要 $N \times 4$ 字节，约 6.56 GB（6.11 GiB）。

2. 一次前向传播的计算量： 输入 1,024 个 token 时，需要做哪些矩阵乘法？总共需要多少 FLOPs?

这里按一条序列计算，只统计矩阵乘法。形状为 (m, n) 和 (n, p) 的两个矩阵相乘，按 $2mnp$ 次浮点运算估算。比如一个 Q 投影，就是 $(1024, 1600)$ 乘 $(1600, 1600)$，需要 $2 \times 1024 \times 1600^2$ FLOPs。

Attention 中还有 $QK^\top$ 和注意力权重乘 V 两次矩阵乘法。每个头的向量长度是 $1600/25 = 64$，合计 25 个头，两次乘法共需 $4 \times 1024^2 \times 1600$ FLOPs。其余 Linear 也按同样的方法计算：

计算部分	整个模型的 FLOPs	约多少万亿次
Q、K、V 和输出投影	$48 \times 8 \times 1024 \times 1600^2$	1.007
$QK^\top$ 、注意力权重乘 V	$48 \times 4 \times 1024^2 \times 1600$	0.322
SwiGLU 的三个 Linear	$48 \times 6 \times 1024 \times 1600 \times 4288$	2.023
LM head	$2 \times 1024 \times 1600 \times 50257$	0.165

相加得到约 3.52 万亿 FLOPs。Embedding 是查表，RMSNorm、RoPE、softmax 等操作不计入这道题的矩阵乘法统计。

3. 计算量主要花在哪里： 哪些模块消耗的 FLOPs 最多？用每一项的 FLOPs 除以总量，就得到占比。SwiGLU 占约 57.5%，消耗最多；Attention 的四个投影占 28.6%，两次注意力矩阵乘法占 9.2%，LM head 占 4.7%。在这个配置下，大部分计算花在 Linear 上。

4. 模型变大后的变化： 换成 small、medium、large 规模，分别计算各模块的 FLOPs 占比，比较哪些占比上升、哪些下降。

继续用上面的算法，替换层数和 `d_model`，词表大小和序列长度保持不变。d_ff 按 $\frac{8}{3} \times d_{model}$ 取最近的 64 的倍数，三个模型分别取 2048、2752、3392。算出各项 FLOPs 后，再除以各自的总量：

模型	总 FLOPs（万亿次）	Q、K、V 和输出投影	两次注意力矩阵乘法	SwiGLU	LM head
small	0.292	19.9%	13.3%	39.8%	27.1%
medium	0.830	24.8%	12.4%	50.1%	12.7%
large	1.769	27.3%	10.9%	54.3%	7.4%
XL	3.517	28.6%	9.2%	57.5%	4.7%

可以看到，模型变大后，投影和 SwiGLU 的占比上升，另外两项的占比下降。因为 `d_ff` 随 `d_model` 增长，Block 中这些 Linear 的计算量大致随 `d_model` 的平方增长；两次注意力矩阵乘法只随它线性增长，LM head 还不随 Block 数量增加。

5. 序列变长后的变化： 把 XL 规模模型的上下文长度从 1,024 增加到 16,384，总 FLOPs 和各模块的占比如何变化？

序列长度变成原来的 16 倍，所有 Linear 的计算量也变成 16 倍；两次注意力矩阵乘法包含序列长度的平方，所以变成 `16^2 = 256` 倍。沿用上面的结果：

$$\text{总 FLOPs} \approx (1.007 + 2.023 + 0.165) \times 16 + 0.322 \times 256 \approx 133.6 \text{ 万亿次}$$

总计算量约为原来的 38 倍。这时，两次注意力矩阵乘法的占比升到 61.7% ，成为最大的计算开销；SwiGLU 占 24.2%，四个投影占 12.1%，LM head 占 2.0%。

06
后记

写到这里，Transformer 的各个模块就都实现了。跟着课程做完这一部分，我们对模型的理解也会提高很多，不过现在我们只是搭好了模型的架子，距离实现一个完整的 LLM 还有一些工作要做，下一篇会继续写训练、推理相关的内容。

参考资料：

CS336: Language Modeling from Scratch

https://cs336.stanford.edu/

Assignment 1: Basics

https://github.com/stanford-cs336/assignment1-basics/tree/main

cs336-assignment（A1 实现）

https://github.com/fengye404/cs336-assignment

Transformer 原论文：Attention Is All You Need

https://arxiv.org/abs/1706.03762

RoPE 原论文：RoFormer: Enhanced Transformer with Rotary Position Embedding

https://arxiv.org/abs/2104.09864

RoPE 推导（苏剑林）：Transformer升级之路：2、博采众长的旋转式位置编码

https://spaces.ac.cn/archives/8265

RMSNorm 原论文：Root Mean Square Layer Normalization

https://arxiv.org/abs/1910.07467

SwiGLU 出处：GLU Variants Improve Transformer

https://arxiv.org/abs/2002.05202


Alibaba Tech
微信公众号
InfoQ
CSDN
头条
今日头条
微博
bilibili
哔哩哔哩
抖音
微信视频号
钉钉视频号
欢迎搜索：阿里技术

欢迎留言一起参与讨论~